

Staff / Senior Machine Learning Engineer

ML & AI-Agent System Design on Google Cloud

Vertex AI · Databricks (Research) · RAG · LLM Agents · Production Readiness

25 ML System Design Questions

20 AI Agent / LLM / RAG Questions

5-Step Framework per Answer

Whiteboard Architectures

Research → Production Boundary

Scope: essential components, infrastructure, data, and the interfaces between research and production environments — not model internals or algorithms.

How to Use This Pack & The Answer Framework

Read this first. It explains how to drive each whiteboard session and what the interviewer at Staff/Senior level is actually scoring.

What this interview tests (per your context)

- **Solution architecture** for ML and AI-Agent systems — components, data flow, interfaces.
- **Production readiness** — scaling, latency, reliability, monitoring, rollback, cost.
- **The research→production boundary** — how a model leaves Databricks experimentation and becomes a governed, monitored Vertex AI service without training-serving skew.
- **Technical leadership** — framing trade-offs, naming risks, sequencing delivery, aligning a team.

You will **not** be asked about model internals or specific algorithms. Keep the conversation on components, infrastructure, data, and interfaces.

The 5-step framework (use it out loud, every time)

Step	What to do	Staff-level signal
1. Clarify Requirements	Functional scope, scale (QPS, data volume), latency SLO, freshness, online vs batch, accuracy vs cost, compliance.	You quantify before you draw.
2. High-Level Design	Draw the end-to-end boxes: data → features → training (Databricks) → registry → serving (Vertex) → monitoring. Name the interfaces.	Clean separation of planes.
3. Deep Dive	Zoom into the 1-2 hardest components for this problem (e.g. feature store, vector index, online/offline parity, drift).	You go deep where it matters.
4. Trade-offs	Make 3-4 explicit choices with the alternative and why (latency vs cost, real-time vs batch, build vs managed).	You reason, you don't recite.
5. Common Mistake	Call out the failure mode that sinks juniors on this exact problem — and how you avoid it.	Hard-won production scars.

Talk track openers (memorize)

- "Before I draw, let me pin three numbers: traffic, latency budget, and data freshness."
- "I'll separate the **research plane** (Databricks) from the **production plane** (Vertex AI) and define the contract between them."
- "The feature pipeline is the same code path offline and online — that's how I avoid training-serving skew."
- "Every model ships behind a monitored endpoint with shadow + canary, and a one-click rollback to the previous registry version."

The Research → Production Boundary (Databricks → Vertex AI)

This is the spine of the whole interview. Reuse this pattern in almost every answer.

Two planes, one contract

Treat **Databricks** as the research/experimentation plane and **Google Cloud / Vertex AI** as the production plane. The interface between them is an explicit, versioned contract — not a hand-off of notebooks.

Research plane · Databricks	Production plane · Google Cloud
Delta Lake / Unity Catalog for curated datasets & lineage	BigQuery + GCS as the governed source of truth
Notebooks, AutoML, distributed training, hyper-parameter search	Vertex AI Training (custom containers) for reproducible re-training
MLflow Tracking — experiments, params, metrics, artifacts	Vertex AI Experiments / Model Registry — promoted candidates
MLflow Model Registry (Staging) + Unity Catalog models	Vertex AI Model Registry (Production) + Artifact Registry image
Offline feature engineering, label generation	Vertex AI Feature Store / Feature Online Store for serving

The promotion pipeline (say this almost verbatim)

- **1. Train & track:** Data scientist trains on Databricks; MLflow logs params, metrics, data snapshot hash, and the model artifact.
- **2. Evaluate & gate:** Candidate must beat the production baseline on a frozen eval set and pass fairness/robustness checks before it can be tagged for promotion.
- **3. Package:** Export the MLflow model, wrap it in a container with a pinned environment, push the image to **Artifact Registry**. Feature transforms ship as shared code so offline == online.
- **4. Register:** Import into **Vertex AI Model Registry** with a version, lineage, and the eval report attached as metadata.
- **5. CI/CD deploy:** **Cloud Build / Cloud Deploy** deploys to a Vertex AI Endpoint as a new version; **shadow** traffic first, then **canary** (e.g. 5%→50%→100%), with automated rollback on metric regression.
- **6. Monitor & loop:** Vertex Model Monitoring watches drift/skew; alerts and labels flow back to Databricks to trigger the next experiment.

Three failure modes this design kills

- **Training-serving skew:** shared feature transformation code + a feature store with point-in-time correctness, so the model sees identical features in both planes.
- **"Works in the notebook" irreproducibility:** pinned containers, data snapshot hashes, and seeds make every promotion reproducible.
- **Silent decay:** drift/skew monitoring + a retraining trigger close the loop instead of letting a stale model rot in production.

Google Cloud + Databricks Service Cheat Sheet

Name the right managed service fast. Map each layer to a default and a fallback.

Layer	Default (managed)	When to swap
Ingestion / streaming	Pub/Sub (events) + Dataflow (Beam) for stream/batch ETL	Kafka on GKE if you need a partitioned log / replay semantics
Batch ETL / big data	Dataflow or Dataproc (Spark); BigQuery for SQL-scale transforms	Databricks jobs when transform logic lives with research
Data warehouse / lake	BigQuery (analytics) + GCS (objects); Delta Lake on Databricks	BigLake to query GCS/Delta in place
Feature store	Vertex AI Feature Store (offline + online serving)	Bigtable/Memorystore for ultra-low-latency custom online store
Experimentation	Databricks notebooks + MLflow Tracking	Vertex AI Workbench for GCP-native teams
Training	Vertex AI Training (custom container, GPU/TPU) ; Databricks for research	GKE + Ray for bespoke distributed jobs
Pipelines / orchestration	Vertex AI Pipelines (KFP) ; Cloud Composer (Airflow)	Databricks Workflows for research-side DAGs
Model registry	Vertex AI Model Registry (prod) ; MLflow Registry (research)	—
Online serving	Vertex AI Endpoints (auto-scaling) ; Cloud Run for light models	GKE for custom runtimes / GPU sharing
Vector search	Vertex AI Vector Search (Matching Engine)	AlloyDB/pgvector or self-hosted (Qdrant/Weaviate) for filters
LLM / GenAI	Vertex AI — Gemini, Model Garden, Agent Builder, RAG Engine	Open models (Llama/Gemma) on Vertex/GKE for control or cost
Low-latency KV / cache	Memorystore (Redis), Bigtable	—
CI/CD	Cloud Build + Cloud Deploy + Artifact Registry	GitHub Actions/GitLab CI calling gcloud
Monitoring	Vertex Model Monitoring + Cloud Monitoring/Logging	Evidently / WhyLabs for richer drift dashboards
Governance	Dataplex + Unity Catalog (lineage, access, catalog)	—

Latency budget reference (quote these)

- Online ML inference SLO: typically p99 < 100–200 ms end-to-end; feature fetch < 20 ms.
- RAG answer: first token < 1 s, full answer 2–6 s; retrieval < 150 ms.
- Streaming feature freshness: seconds (Dataflow) vs hours (batch) — pick per use case.
- Batch scoring: throughput-bound, run on a schedule, write to BigQuery/Bigtable for lookup.

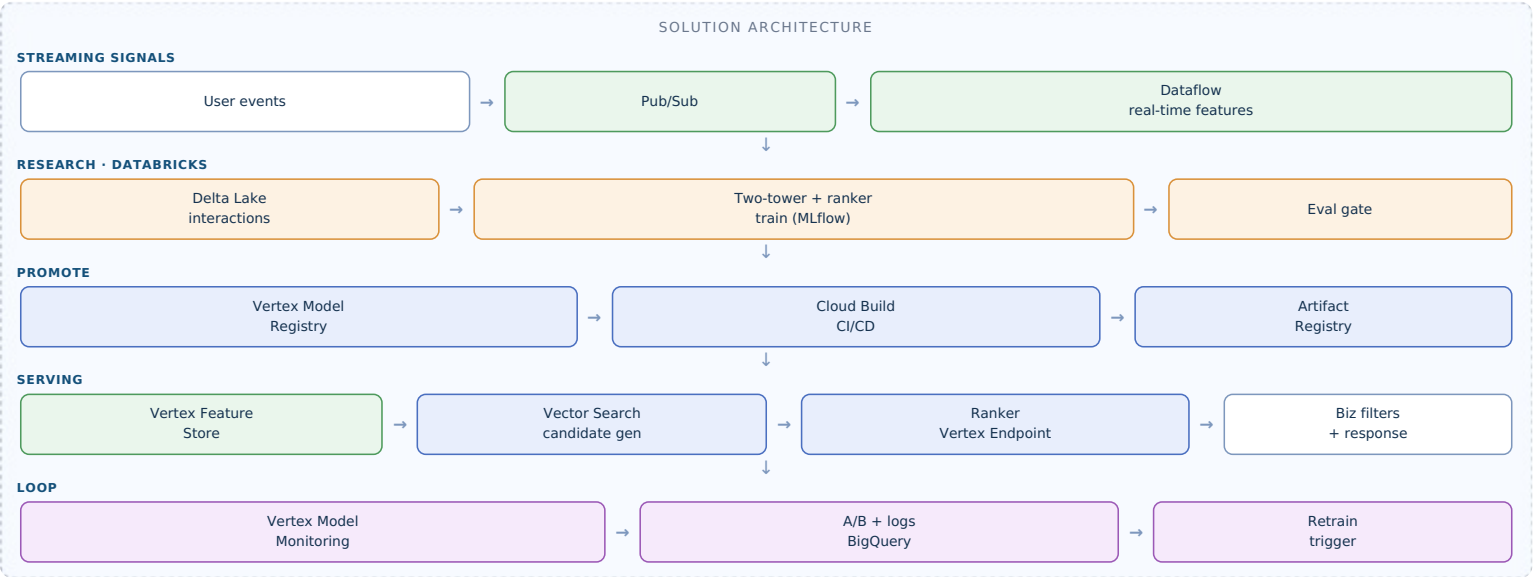
Section A · ML System Design — 25 Questions

Production ML architecture on Google Cloud (Vertex AI) with Databricks as the research/experimentation plane. Each answer follows the 5-step interview framework and includes a whiteboard-style architecture.

ML QUESTION 1

Design a real-time product recommendation system for an e-commerce platform

Focus: Recommenders, candidate generation + ranking, online serving



Step 1 — Clarify Requirements

Pin scope before drawing. **Traffic:** ~10k QPS at peak, p99 latency budget < 150 ms. **Catalog:** tens of millions of items, millions of daily-active users. **Objective:** optimize for add-to-cart / revenue, not just clicks. **Freshness:** session signals must be reflected within seconds; batch user embeddings can refresh nightly. **Surfaces:** homepage, PDP, cart. Confirm cold-start (new users/items) is in scope and whether we need explainability for merchandising.

Step 2 — High-Level Design

Use the industry-standard **two-stage** pattern. **Candidate generation** narrows millions of items to a few hundred via approximate nearest-neighbour over embeddings; **ranking** scores those with a heavier model using rich features. Offline, two-tower / sequence models are trained on **Databricks**; item embeddings land in **Vertex AI Vector Search** and features in **Vertex AI Feature Store**. A serving service on Vertex Endpoints orchestrates retrieve→rank→filter (business rules, inventory). Streaming session events flow through **Pub/Sub** → **Dataflow** to update real-time features.

Step 3 — Deep Dive

Feature parity: the same transformation code computes features offline (training) and online (serving) to kill skew. **Candidate gen:** ANN index served by Vector Search, refreshed as item embeddings re-compute; multiple retrievers (collaborative, content, trending) are blended. **Ranking:** gradient-boosted or wide-and-deep model fed user, item, and context features; fetch from the online store in <20 ms. **Cold start:** content-based embeddings + popularity priors until interaction data accrues. **Loop:** impressions/clicks logged to BigQuery feed the next training run and online evaluation.

Step 4 — Trade-offs to Discuss

- **Two-stage vs single model:** two-stage scales to huge catalogs cheaply; loses some accuracy a single ranker would capture — worth it at this catalog size.
- **Real-time vs batch features:** session features need streaming (cost, complexity); long-term profile features can be nightly batch.
- **Managed Vector Search vs self-hosted:** managed for ops simplicity; self-host only if you need exotic filtering or cost control at extreme scale.
- **Precompute vs on-the-fly:** precompute homepage recs for heavy users to cut latency; compute live for the long tail.

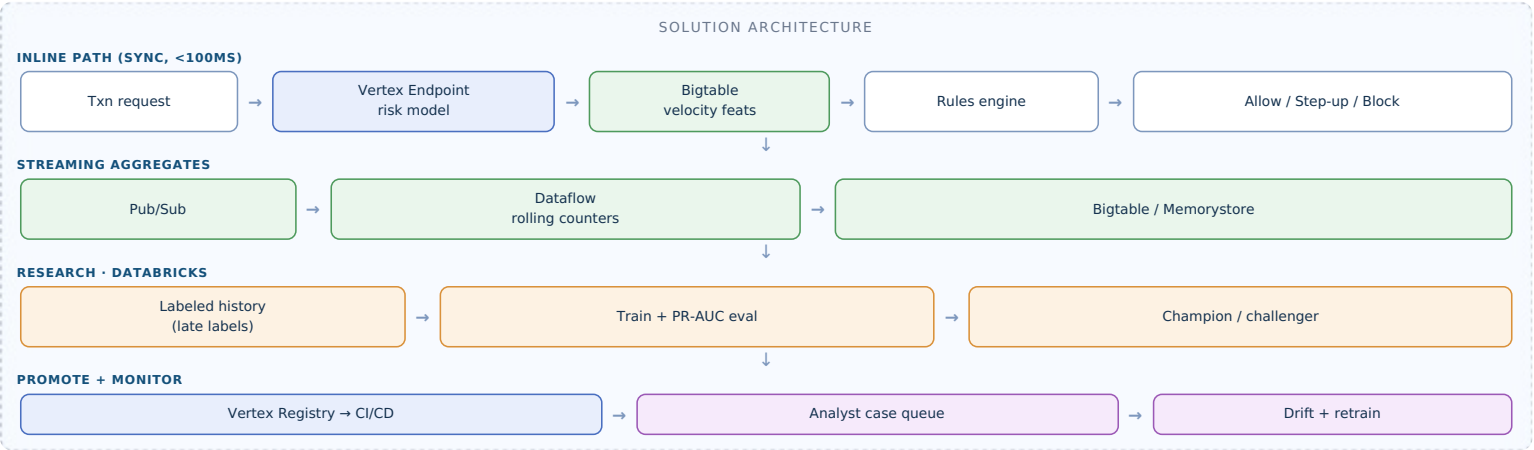
Step 5 — Common Mistake

Optimizing the model offline (NDCG/AUC) and assuming it lifts revenue online. Offline gains often don't transfer due to **feedback loops and position bias**. The fix: design the A/B framework and online metrics *first*, log propensities, and treat the offline model as a candidate that must prove itself in a canary. Also: forgetting business filters (out-of-stock, region) at serving time produces embarrassing recs.

ML QUESTION 2

Design a real-time fraud / transaction risk detection system

Focus: Low-latency classification, streaming features, imbalanced data, human-in-the-loop



Step 1 — Clarify Requirements

Latency: inline scoring must return in <50-100 ms so it can block a payment. **Volume:** thousands of transactions/sec, highly imbalanced (<1% fraud). **Cost asymmetry:** a missed fraud (FN) vs a blocked good customer (FP) have very different costs — clarify the target precision/recall operating point. **Freshness:** velocity features (txns in last minute) must be near-instant. **Compliance:** decisions must be explainable/auditable; data residency may apply. Confirm whether we also need an offline case-management queue for analysts.

Step 2 — High-Level Design

Inline synchronous scoring path plus an asynchronous learning loop. Transactions hit a low-latency **Vertex Endpoint** (or Cloud Run) that fetches velocity/aggregate features from a fast online store (**Bigtable/Memorystore**) and returns a risk score → allow / challenge (step-up auth) / block. A **Pub/Sub → Dataflow** stream maintains rolling aggregates. Models train on **Databricks** over historical labeled data (labels arrive late via chargebacks). A rules engine runs alongside the model for hard policies. Flagged cases route to an analyst queue whose decisions become new labels.

Step 3 — Deep Dive

Late labels: ground truth (chargebacks) can arrive 30-90 days later, so training data must be assembled with point-in-time joins to avoid leakage. **Imbalance:** handle with class weighting / focal loss and evaluate on PR-AUC and precision@operating-recall, never raw accuracy. **Velocity features:** Dataflow keeps per-card/device/IP counters in Bigtable with TTLs. **Explainability:** attach reason codes (e.g. SHAP) to each decision for audit and customer disputes. **Adversarial drift:** fraudsters adapt, so monitoring and fast retraining/champion-challenger are mandatory.

Step 4 — Trade-offs to Discuss

- **Model vs rules:** rules are transparent and instant for known patterns; ML catches novel fraud. Run both — rules as guardrails, model for nuance.
- **Block vs challenge:** blocking maximizes safety but hurts conversion; step-up auth balances both.
- **Threshold tuning:** a single global threshold is simple but a segment-aware threshold reduces FP on trusted cohorts.
- **Sync feature fetch latency vs richness:** cap online features to those fetchable in <15 ms.

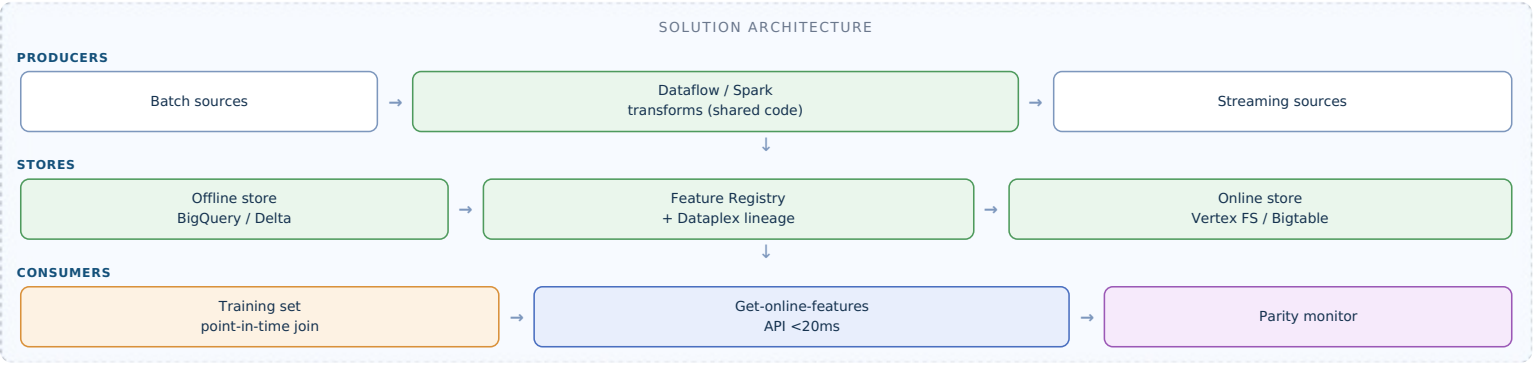
Step 5 — Common Mistake

Evaluating on accuracy with a 99% non-fraud base rate — a model that predicts "never fraud" looks 99% accurate and is useless. Equally common: **data leakage** from using post-event fields (e.g. final chargeback status) as features. Always reconstruct features as they existed at decision time, and report precision/recall at the chosen operating point with the business cost matrix.

ML QUESTION 3

Design a centralized Feature Store for an ML platform

Focus: Feature platform, offline/online parity, point-in-time correctness, governance



Step 1 — Clarify Requirements

Who are the users — many teams sharing features, or one? **Requirements:** offline store for training (large historical joins) and online store for low-latency serving (<20 ms). **Freshness tiers:** batch (daily), streaming (seconds), on-demand (computed at request). **Scale:** thousands of features, billions of rows. **Governance:** discovery, ownership, lineage, access control, PII handling. Confirm the non-negotiable: **point-in-time correctness** for training labels and **offline/online parity** so the same value is served as was trained on.

Step 2 — High-Level Design

A dual-store design. **Offline store** (BigQuery / Delta Lake) holds full history for training-time point-in-time joins; **online store** (Vertex AI Feature Store online / Bigtable) holds the latest value per entity for serving. A single **transformation layer** (shared library + Dataflow/Spark jobs) writes both, guaranteeing parity. A **registry/catalog** (Vertex Feature Store metadata + Dataplex) tracks feature definitions, owners, freshness, and lineage. Producers register features; consumers fetch training sets offline and feature vectors online by entity key.

Step 3 — Deep Dive

Point-in-time joins: when building a training set, each label row must join only feature values known *before* the label timestamp — otherwise leakage. The offline store keeps versioned timestamps to support this. **Parity:** the same transformation code path materializes offline and online; reconciliation jobs detect divergence. **Freshness:** streaming features via Dataflow upsert into the online store; batch via scheduled jobs. **Serving:** a get-online-features API returns a vector by entity id in single-digit ms. **Governance:** PII tagging, row/column ACLs, and lineage so a model can be traced to exact feature versions.

Step 4 — Trade-offs to Discuss

- **Buy (Vertex Feature Store) vs build (Bigtable + custom):** managed accelerates delivery; build only for extreme latency/cost or special semantics.
- **Materialize vs on-demand:** precomputing is fast but storage-heavy and can be stale; on-demand is fresh but adds serving latency.
- **One big store vs per-team stores:** central enables reuse and governance but adds a dependency and coordination cost.
- **Streaming everywhere vs tiered freshness:** stream only features that need it — streaming is the expensive part.

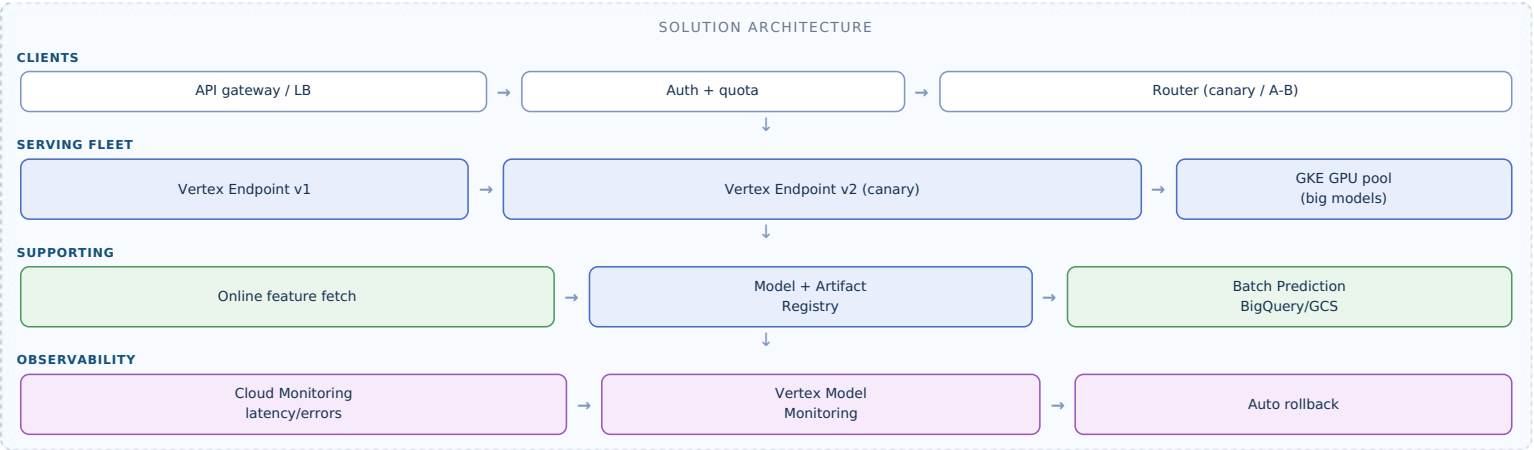
Step 5 — Common Mistake

Building separate, divergent pipelines for training and serving — the #1 cause of **training-serving skew**, where a model performs great offline and silently degrades online. The cure is a single transformation definition that materializes both stores, plus parity monitoring. The second classic mistake is ignoring point-in-time correctness and leaking future information into training labels.

ML QUESTION 4

Design a scalable model serving / inference platform (ML-as-a-Service)

Focus: Serving infrastructure, autoscaling, multi-model, latency, GPU efficiency



Step 1 — Clarify Requirements

Is this one model or a platform hosting many models for many teams? **SLOs:** p99 latency target, availability (e.g. 99.9%), traffic shape (steady vs spiky). **Model types:** CPU tree models, deep nets, GPU models, large transformers? **Modes:** real-time, batch, streaming. **Requirements:** versioning, canary, A/B, multi-tenant isolation, cost efficiency on GPUs. Confirm whether request payloads are small (tabular) or large (images), which changes the network and batching design.

Step 2 — High-Level Design

A managed serving layer on **Vertex AI Endpoints** (or GKE for custom runtimes) fronted by an API gateway / load balancer. Each model version is a container image from **Artifact Registry**, registered in **Vertex Model Registry**. Endpoints autoscale on QPS/latency. A routing layer supports traffic splitting for canary/A-B. Batch jobs use Vertex Batch Prediction reading/writing BigQuery/GCS. A feature-fetch sidecar pulls online features. Observability (latency, errors, throughput, drift) is centralized in Cloud Monitoring + Vertex Model Monitoring.

Step 3 — Deep Dive

GPU efficiency: dynamic request batching and concurrency tuning to keep accelerators busy; consider model server (Triton/TF-Serving) for batching and multi-model on one GPU. **Autoscaling:** scale on custom latency metrics, not just CPU; keep warm replicas to avoid cold starts. **Multi-tenant isolation:** per-team quotas, separate node pools for noisy neighbors. **Versioning & rollout:** registry-driven deploys, shadow then canary with automated rollback on SLO breach. **Payload handling:** stream/resize large inputs; cache repeated requests.

Step 4 — Trade-offs to Discuss

- **Managed Endpoints vs GKE:** managed minimizes ops; GKE gives control over GPU sharing, custom runtimes, and cost.
- **Real-time vs batch:** batch is far cheaper for non-interactive scoring; reserve real-time for user-facing latency.
- **One endpoint per model vs multi-model server:** multi-model improves GPU utilization but couples deploys.
- **Always-warm vs scale-to-zero:** warm replicas cost money but remove cold-start latency tails.

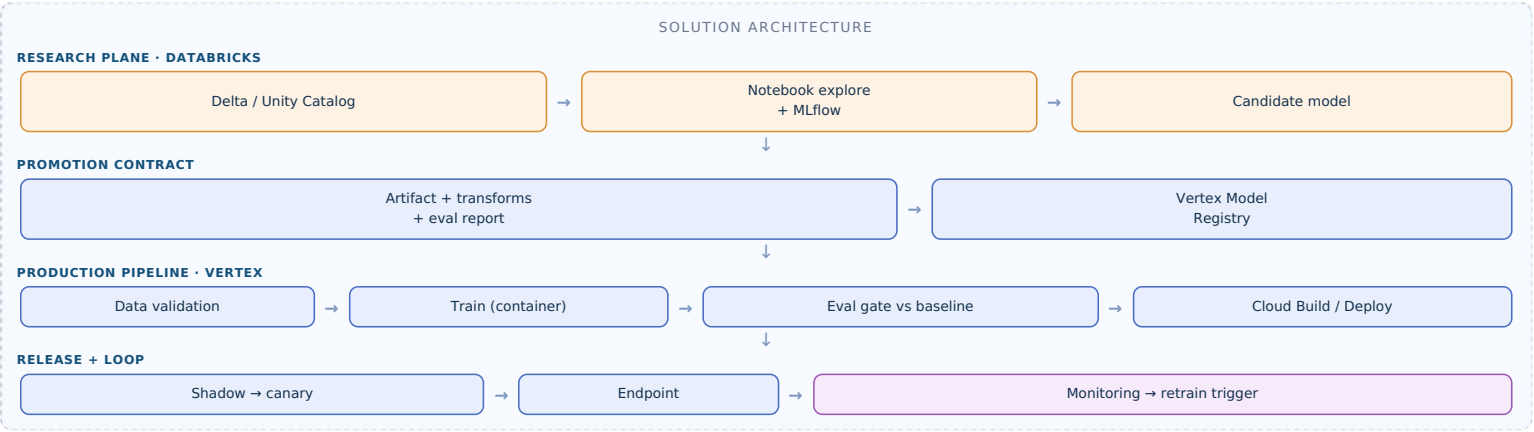
Step 5 — Common Mistake

Autoscaling on CPU when the real constraint is latency or GPU memory — the fleet scales late and p99 blows up under spikes. Also under-provisioning warm capacity, causing cold-start latency cliffs. Fix: scale on request-latency / queue-depth signals, keep a warm floor, and load-test the spiky traffic shape before launch rather than the average.

ML QUESTION 5

Design an end-to-end training pipeline from research to production

Focus: MLOps, reproducibility, CI/CD for models, Databricks-to-Vertex promotion



Step 1 — Clarify Requirements

What triggers training — schedule, data volume, or drift alert? **Scale:** data size, GPU needs, training duration. **Reproducibility bar:** must we recreate any past model bit-for-bit? **Governance:** approval gates, audit, model cards. **Cadence:** daily retrain vs ad-hoc research. Clarify the split: ad-hoc exploration lives in **Databricks**; the productionized, automated retraining must run as a governed Vertex pipeline. Confirm rollback and shadow requirements.

Step 2 — High-Level Design

Two coupled pipelines. **Research:** Databricks notebooks + MLflow for exploration, with curated data in Delta/Unity Catalog. **Production:** a **Vertex AI Pipeline** (KFP) that does data validation → feature build → train (custom container) → evaluate → gate → register. Promotion moves the winning MLflow model into Vertex Model Registry; **Cloud Build / Cloud Deploy** ships it to an Endpoint with shadow+canary. Everything is parameterized by a data snapshot hash and pinned container for reproducibility.

Step 3 — Deep Dive

Reproducibility: pin container images, library versions, random seeds, and reference an immutable data snapshot (Delta time-travel / BigQuery snapshot). **Validation gates:** schema + distribution checks on input data, then an eval gate where the candidate must beat the production baseline on a frozen test set plus fairness/robustness checks. **Promotion contract:** the artifact, eval report, feature transform code, and lineage travel together. **CD:** registry event triggers deploy; shadow validates parity on live traffic before canary. **Loop:** monitoring alerts re-enter the research plane.

Step 4 — Trade-offs to Discuss

- **Scheduled vs triggered retraining:** scheduled is predictable; drift-triggered saves compute and reacts faster but is harder to reason about.
- **Full automation vs human gate:** auto-promotion is fast but risky for high-stakes models; add a manual approval for regulated cases.
- **Databricks training vs Vertex training:** research velocity on Databricks vs governed reproducibility on Vertex — many teams research on Databricks, retrain on Vertex.
- **Snapshot vs live data for eval:** frozen eval set ensures comparability across versions.

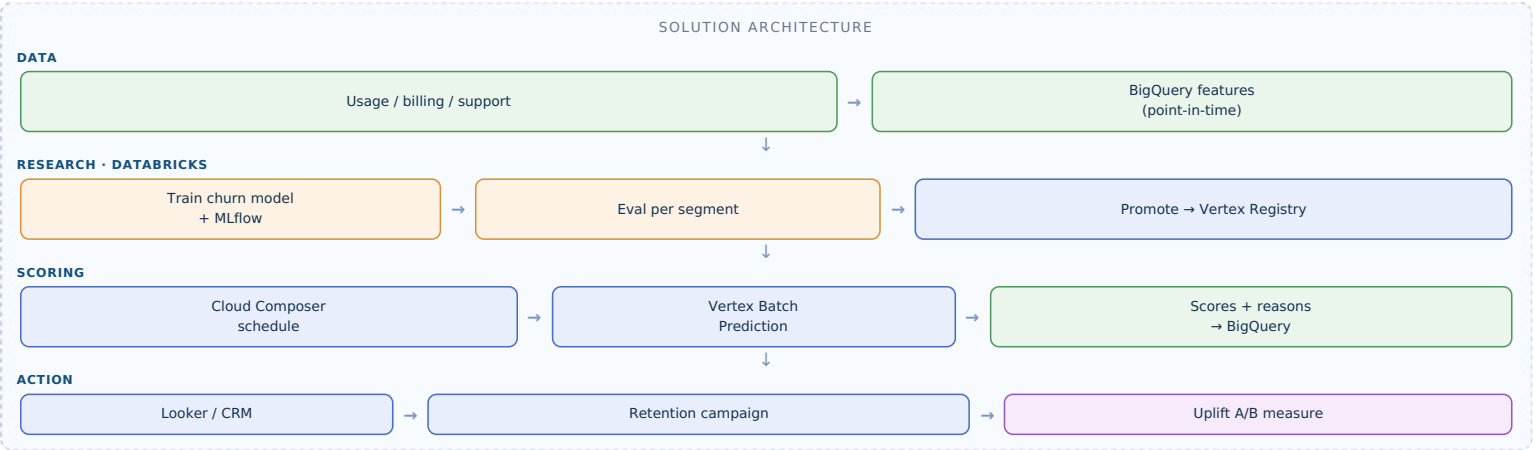
Step 5 — Common Mistake

Letting a notebook model go straight to production by copy-pasting code — nothing is pinned, nothing is reproducible, and the feature logic silently diverges from serving. The professional pattern is a **promotion pipeline with a contract:** artifact + transforms + eval + lineage, an eval gate against the live baseline, and shadow before canary. Never promote on offline metrics alone.

ML QUESTION 6

Design a customer churn prediction system

Focus: Batch scoring, label definition, actionability, segment analysis



Step 1 — Clarify Requirements

What is 'churn' precisely — cancellation, N days inactive, downgrade? Define the **prediction window** (churn in next 30/60/90 days) and **cadence** (daily/weekly batch is usually enough; real-time rarely needed). **Action:** who consumes scores — a retention campaign, a CSM dashboard? That determines whether we need reason codes and ranked intervention lists. **Scale:** millions of accounts. Confirm we measure business uplift (retained revenue), not just model AUC.

Step 2 — High-Level Design

A scheduled **batch scoring** pipeline. Features are built from product usage, billing, and support data in **BigQuery**; a model trained on **Databricks** is promoted to **Vertex** and run via Vertex Batch Prediction on a schedule (Cloud Composer). Scores + reason codes write back to BigQuery and surface in Looker / the CRM, optionally pushing high-risk accounts into a campaign tool. The training set uses point-in-time features and a churn label defined over the prediction window.

Step 3 — Deep Dive

Label leakage: features must reflect only what was known before the prediction window; e.g. don't use 'account closed' fields. **Window framing:** snapshot features at time T, label = churned within [T, T+90]. **Actionability:** rank accounts by expected value (risk x account value) and attach drivers (SHAP) so CSMs know *why* and *what to do*. **Segments:** evaluate per cohort to avoid a model that only works on the majority segment. **Uplift:** ideally model treatment effect, not just risk, to target saveable customers.

Step 4 — Trade-offs to Discuss

- **Batch vs real-time:** churn evolves slowly — daily/weekly batch is cheaper and sufficient.
- **Risk model vs uplift model:** risk is simpler; uplift targets customers who are actually persuadable, improving campaign ROI.
- **Accuracy vs explainability:** retention teams need reasons, favoring interpretable or SHAP-explained models.
- **Global vs per-segment models:** segment models capture heterogeneity but multiply maintenance.

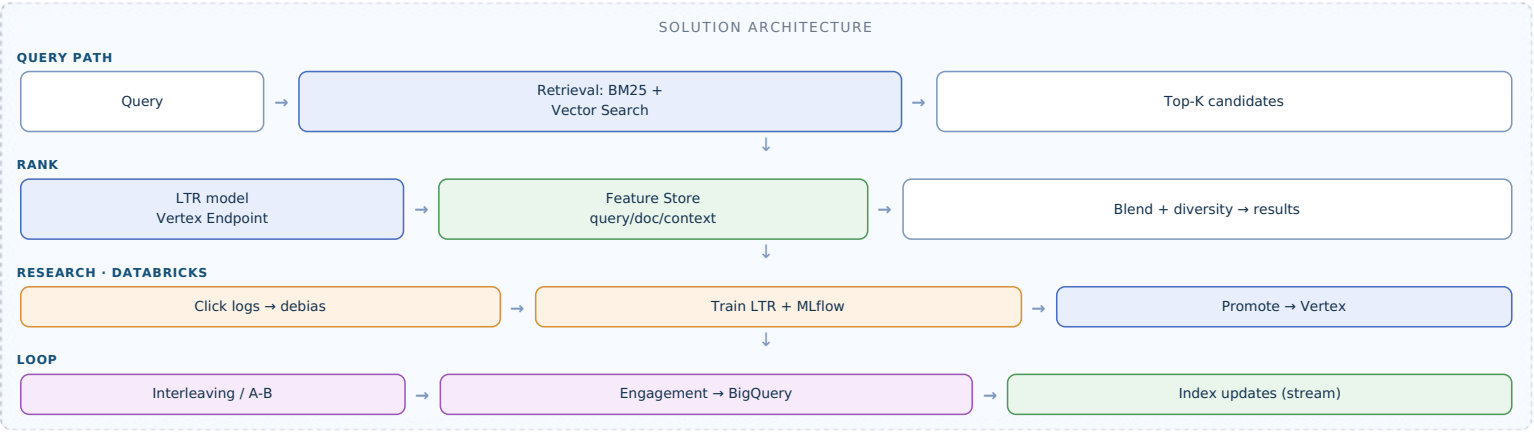
Step 5 — Common Mistake

Predicting churn that no one can act on — flagging customers who already left, or with no intervention attached. High AUC is worthless if it doesn't change retention. Tie the model to a workflow, prioritize by expected value, and validate with a holdout/A-B on the actual retention campaign. Also beware leakage from cancellation-adjacent fields inflating offline metrics.

ML QUESTION 7

Design a search ranking system (learning-to-rank)

Focus: Retrieval + ranking, relevance, click models, online metrics



Step 1 — Clarify Requirements

Corpus & query volume: document count, QPS, p99 latency budget (<200 ms typical). **Relevance signal:** do we have human judgments, or only clicks (biased)? **Objective:** relevance, engagement, or revenue blend? **Personalization:** is user context in scope? **Freshness:** how fast must new documents become searchable? Confirm we'll measure with online metrics (CTR, success rate) plus offline NDCG, and that position/presentation bias must be handled.

Step 2 — High-Level Design

Classic **retrieval → ranking** funnel. Retrieval pulls candidates via inverted index (BM25) and/or semantic embeddings (**Vertex Vector Search**). A learning-to-rank model (trained on **Databricks**, served on **Vertex**) re-scores the top-K using query, document, and context features from the feature store. A blending layer applies business boosts and diversity. Click/engagement logs flow to BigQuery to train the next ranker. Index updates stream in as documents change.

Step 3 — Deep Dive

Position bias: clicks are biased by rank; train with inverse-propensity weighting or a click model so the ranker learns true relevance, not just 'top results get clicked.' **Hybrid retrieval:** combine lexical (exact match, rare terms) and semantic (synonyms, intent) for recall. **Feature freshness:** query-time features (e.g. query embedding) computed live; document features precomputed. **Latency:** retrieve a few hundred, rank with a model cheap enough for the budget; cache hot queries. **Eval:** interleaving experiments give sensitive online comparisons.

Step 4 — Trade-offs to Discuss

- **Lexical vs semantic retrieval:** lexical nails exact terms; semantic handles intent. Hybrid wins but adds cost.
- **Heavier ranker vs latency:** deeper models lift relevance but cost ms; cascade rankers (cheap then expensive) balance both.
- **Clicks vs human labels:** clicks are abundant but biased; human judgments are clean but expensive.
- **Personalized vs global ranking:** personalization helps engagement but complicates caching and evaluation.

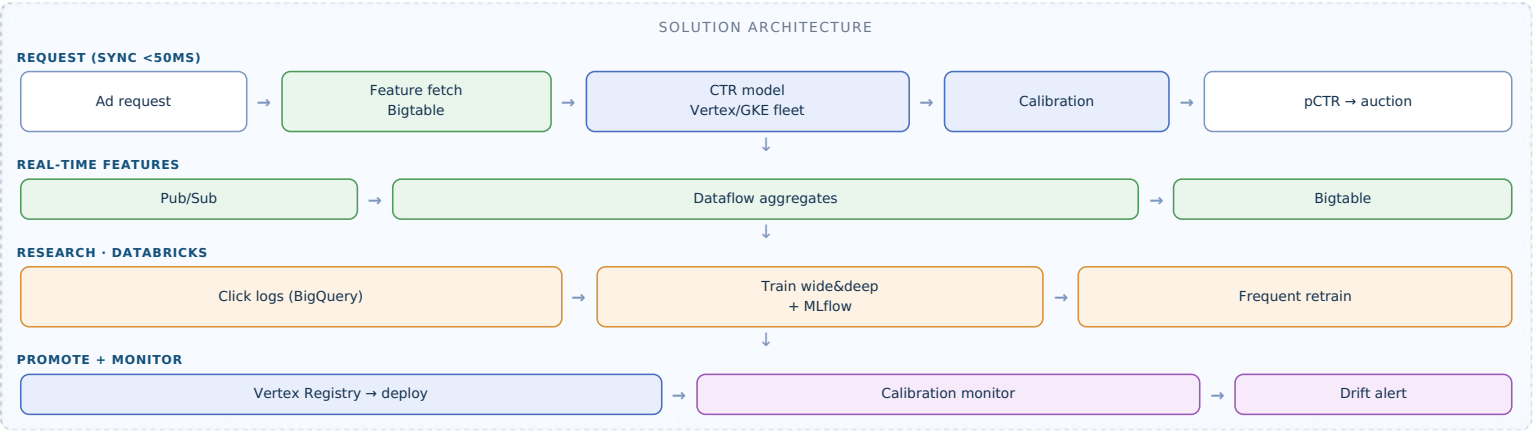
Step 5 — Common Mistake

Training directly on raw clicks without correcting for position bias, so the model just learns to reproduce the current ranking and never improves. Use propensity correction or a click model, and validate with interleaving / A-B rather than offline NDCG alone. Also: ignoring the long tail of rare queries where semantic retrieval matters most.

ML QUESTION 8

Design an ad click-through-rate (CTR) prediction system

Focus: High-QPS prediction, sparse features, calibration, real-time bidding adjacency



Step 1 — Clarify Requirements

Scale: very high QPS (100k+), strict latency (<50 ms) since CTR feeds an auction. **Features:** massive sparse/categorical space (user, ad, context, cross features). **Calibration:** predicted probability must be well-calibrated because it multiplies into bid price (pCTR x bid). **Freshness:** new ads/users appear constantly — cold start and fast online updates matter. Confirm we optimize for calibrated probability and revenue, not just ranking AUC.

Step 2 — High-Level Design

A high-throughput, low-latency scoring service. Sparse features hashed/embedded; a wide-and-deep or factorization-style model (trained on **Databricks**, possibly with frequent online updates) served on a horizontally scaled fleet (**Vertex Endpoints / GKE**). Real-time features (recent user activity, ad frequency) maintained via **Pub/Sub → Dataflow** into **Bigtable**. A calibration layer adjusts raw scores. Logs stream to BigQuery for near-continuous retraining.

Step 3 — Deep Dive

Calibration: apply Platt/isotonic calibration and monitor calibration error continuously, because the auction trusts the probability. **Sparse features:** feature hashing + learned embeddings keep the space tractable; manage embedding tables and cold IDs. **Freshness:** incremental/online learning or frequent mini-batch retrains keep up with new ads and trends. **Latency:** precompute embeddings, fetch features in <10 ms, keep the model lean. **Scale:** shard the fleet, cache hot ads, and load-test the peak.

Step 4 — Trade-offs to Discuss

- **Model complexity vs latency/QPS:** at 100k QPS, a slightly worse but 5x cheaper model can win on total revenue.
- **Online vs batch updates:** online learning reacts fast but is operationally fragile; frequent batch retrains are a safer middle ground.
- **Calibration method:** isotonic is flexible but can overfit; Platt is stable.
- **Embedding size:** bigger embeddings lift accuracy but blow up memory and latency.

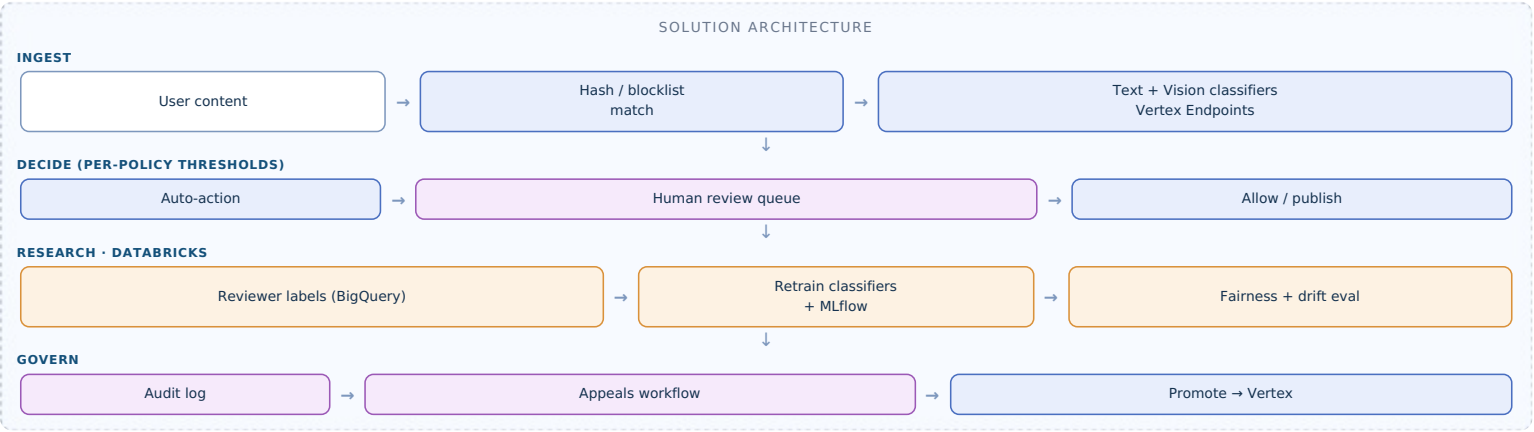
Step 5 — Common Mistake

Optimizing AUC/ranking and ignoring **calibration** — an uncalibrated pCTR that feeds bid prices systematically over- or under-bids and burns budget even if ranking is perfect. Monitor calibration as a first-class metric. Second mistake: neglecting cold-start for fresh ads, which never accumulate clicks if the model never surfaces them (exploration is needed).

ML QUESTION 9

Design a content moderation system (text/image abuse detection)

Focus: Multi-modal classification, human review, policy precision/recall, appeals



Step 1 — Clarify Requirements

Modalities: text, images, video? **Volume & latency:** moderate at upload (near real-time) vs proactive sweeps (batch). **Policy:** categories (hate, violence, CSAM, spam) with very different cost asymmetries — some require near-zero false negatives. **Human-in-the-loop:** what gets auto-actioned vs queued for review? **Appeals & audit:** decisions must be logged and reversible. Confirm legal/compliance constraints and that severe categories route to specialized handling.

Step 2 — High-Level Design

A tiered pipeline. Content hits classifiers (text + vision models, trained on **Databricks**, served on **Vertex Endpoints**) producing per-policy scores. A **decision layer** applies thresholds: high-confidence violations auto-action, ambiguous cases queue for **human review**, clear-pass content publishes. Reviewer decisions feed back as labels. Matched-hash systems catch known bad content instantly. Everything logs to BigQuery for audit, metrics, and appeals.

Step 3 — Deep Dive

Threshold per policy: each category gets its own operating point reflecting its cost asymmetry (e.g. high recall for severe harm, high precision for borderline). **Human-in-the-loop:** route the uncertain band to reviewers; their labels continuously improve the model and adapt to new abuse patterns. **Multi-modal:** combine signals (image + caption + user history). **Adversarial drift:** bad actors evolve; monitor and retrain frequently, keep a fast-path blocklist/hash store. **Fairness:** audit for biased enforcement across groups.

Step 4 — Trade-offs to Discuss

- **Auto-action vs human review:** automation scales but errs; humans are accurate but slow/costly. Tune the review band by category risk.
- **Precision vs recall:** severe categories favor recall (catch everything) accepting more FPs; spam favors precision.
- **Real-time vs batch:** moderate at upload for harmful content; batch-sweep for lower-severity at scale.
- **One multi-label model vs per-policy models:** per-policy gives independent control; multi-label is cheaper.

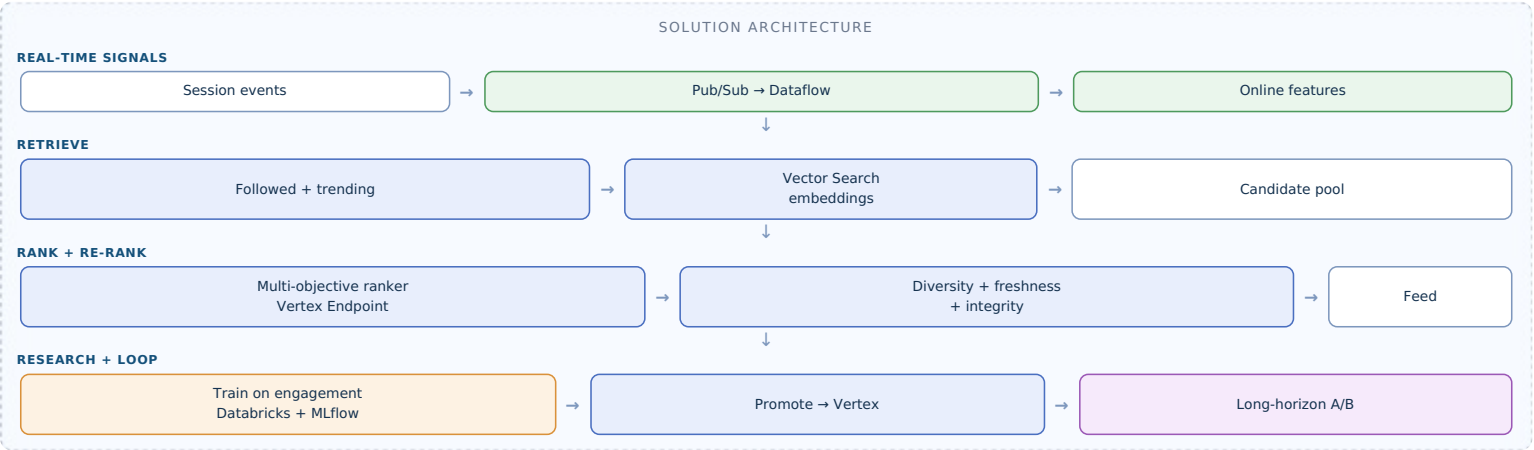
Step 5 — Common Mistake

Using a single global threshold across all policies, which is wrong for every category at once and produces either dangerous false negatives or mass false positives. Each policy needs its own operating point and review routing. Also: shipping without an appeals + audit trail, which is both an ethical and legal liability, and forgetting that moderation is adversarial and decays without constant retraining.

ML QUESTION 10

Design a personalized news / content feed ranking system

Focus: Recommenders, real-time signals, multi-objective ranking, freshness



Step 1 — Clarify Requirements

Objective blend: engagement, dwell time, diversity, and freshness — pure click-optimization causes clickbait and filter bubbles, so clarify the multi-objective target. **Scale:** millions of users, fast-changing content (news decays in hours). **Latency:** feed must assemble in <200 ms. **Freshness:** brand-new content must be rankable immediately (cold start). Confirm we need real-time session signals and guardrails against integrity issues (misinfo, over-personalization).

Step 2 — High-Level Design

Two-stage retrieve+rank with a multi-objective ranker. Candidate generation gathers fresh + relevant items (followed sources, embeddings via **Vector Search**, trending). The ranker (trained on **Databricks**, served on **Vertex**) scores a blended objective using user, item, and real-time session features. A re-ranking layer enforces diversity, freshness decay, and integrity filters. Session events stream via **Pub/Sub -> Dataflow** to update features in seconds. Engagement logs train the next model.

Step 3 — Deep Dive

Multi-objective: combine predicted engagement, dwell, and diversity via a weighted or constrained objective; weights tuned by online experiments to business goals. **Freshness:** apply time-decay and ensure new content can be retrieved before it has interaction history (content embeddings + exploration). **Real-time signals:** recent session behavior strongly predicts next click; maintain it in a fast online store. **Integrity:** down-rank or filter low-quality/misinfo with dedicated classifiers. **Exploration:** inject some novelty to avoid feedback loops collapsing the feed.

Step 4 — Trade-offs to Discuss

- **Engagement vs long-term value:** maximizing clicks short-term can harm retention/trust; optimize a blended, long-horizon objective.
- **Personalization vs diversity:** too much personalization creates bubbles; enforce diversity constraints.
- **Freshness vs relevance:** news needs recency weighting that evergreen content doesn't.
- **Exploration vs exploitation:** exploration costs short-term engagement but keeps the model healthy.

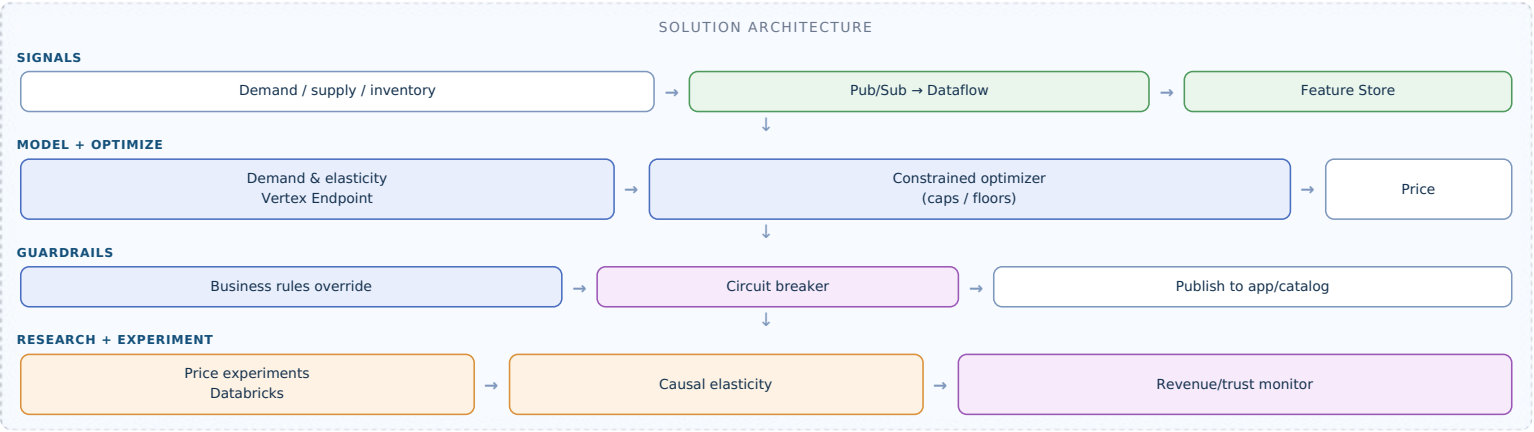
Step 5 — Common Mistake

Optimizing a single engagement metric (CTR) and getting a clickbait, low-diversity feed that wins this week and loses retention next quarter. Staff-level answer: define a multi-objective with explicit diversity/ integrity guardrails, measure long-horizon outcomes, and add exploration to break feedback loops. Also: failing to handle freshness/cold start so new content never surfaces.

ML QUESTION 11

Design a dynamic pricing system

Focus: Forecasting + optimization, business constraints, experimentation, guardrails



Step 1 — Clarify Requirements

What are we pricing — rides, hotel rooms, retail SKUs? **Objective:** revenue, margin, utilization, or fairness-constrained? **Cadence:** real-time surge vs periodic repricing. **Constraints:** legal limits, price-change frequency, competitor matching, brand perception. **Inputs:** demand, supply, inventory, elasticity. Confirm guardrails (caps/floors) and that we measure causal impact via experiments, since naive pricing models can spiral.

Step 2 — High-Level Design

Two layers: a **demand/elasticity model** and a **pricing optimizer under constraints**. Demand forecasting models (trained on **Databricks**, served on **Vertex**) estimate demand and price sensitivity; an optimization layer chooses the price maximizing the objective subject to caps, floors, and inventory. Real-time signals (current demand/supply) stream via **Pub/Sub → Dataflow**. Prices publish to the app/catalog. A pricing experimentation platform measures elasticity and validates changes.

Step 3 — Deep Dive

Elasticity estimation is causal, not predictive: you must experiment (randomized price tests / holdouts) to learn how demand responds to price, otherwise you fit correlations and mis-price. **Optimization:** constrained optimization (caps/floors, change-rate limits) prevents extreme or oscillating prices. **Guardrails:** hard business rules override model output; circuit breakers stop runaway pricing. **Freshness:** surge contexts need near-real-time demand signals. **Monitoring:** watch realized revenue/margin and customer-trust metrics, not just model error.

Step 4 — Trade-offs to Discuss

- **Real-time surge vs stable pricing:** dynamic pricing maximizes short-term revenue but can erode trust; balance with caps and smoothing.
- **Predictive elasticity vs experimental:** experiments are costly but the only trustworthy way to learn causal response.
- **Optimize revenue vs margin vs utilization:** these conflict; the objective must be a deliberate business choice.
- **Rule-based guardrails vs pure model:** always keep guardrails — pure model pricing fails unsafely.

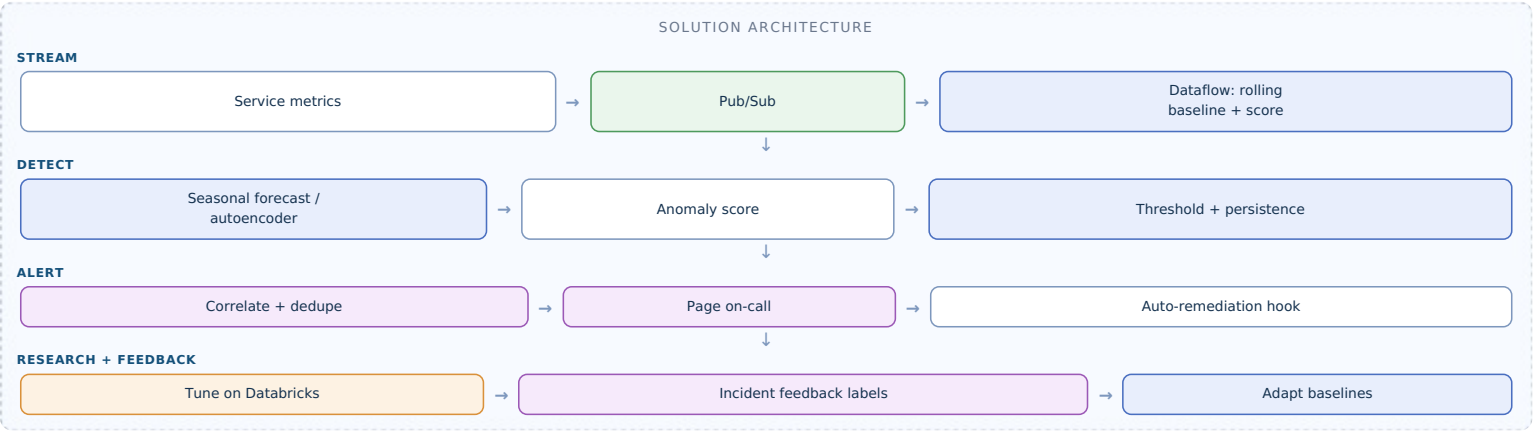
Step 5 — Common Mistake

Treating elasticity as a prediction problem and learning from observational data, which confounds price with demand and yields a model that recommends harmful prices confidently. You must run controlled price experiments to estimate causal elasticity. Also: shipping without caps/floors and circuit breakers, allowing a model bug to produce absurd prices and PR disasters.

ML QUESTION 12

Design an anomaly detection system for infrastructure / operational metrics

Focus: Time-series, streaming, alerting, low false-positive design



Step 1 — Clarify Requirements

What signals: thousands of metrics (latency, error rate, throughput) across services. **Latency:** detect within seconds-to-minutes. **Labels:** usually unlabeled — mostly unsupervised. **Cost asymmetry:** alert fatigue (too many FPs) is the killer; missed real incidents are costly too. **Seasonality:** daily/weekly patterns. Confirm the consumer (on-call alerting, auto-remediation) and that we need explainability ('which metric, how anomalous').

Step 2 — High-Level Design

A streaming detection pipeline. Metrics flow through **Pub/Sub** → **Dataflow**, which computes rolling baselines and scores deviations using statistical + ML detectors (seasonal decomposition, forecast-error, or autoencoders). Models/baselines are developed on **Databricks** and deployed as streaming logic or **Vertex** endpoints. An alerting layer with deduplication/correlation reduces noise and pages on-call. Detected anomalies and feedback (was it a real incident?) feed model tuning.

Step 3 — Deep Dive

Seasonality: models must account for daily/weekly cycles or they alarm every morning peak; seasonal-aware forecasting or decomposition handles this. **Alert fatigue:** correlate related alerts, suppress duplicates, and require sustained deviation before paging. **Adaptivity:** baselines must adapt to deploys and gradual changes without masking real regressions. **Explainability:** surface the offending metric and magnitude so on-call can act fast. **Scale:** per-metric lightweight detectors plus cross-metric correlation for incidents.

Step 4 — Trade-offs to Discuss

- **Sensitivity vs alert fatigue:** the central trade-off — too sensitive and on-call ignores alerts; too lax and incidents slip.
- **Statistical vs ML detectors:** simple statistical methods are robust and explainable; ML catches complex patterns but is opaque and needs tuning.
- **Per-metric vs holistic models:** per-metric scales and is simple; holistic finds correlated incidents.
- **Static vs adaptive baselines:** adaptive avoids deploy-driven FPs but can hide slow drifts.

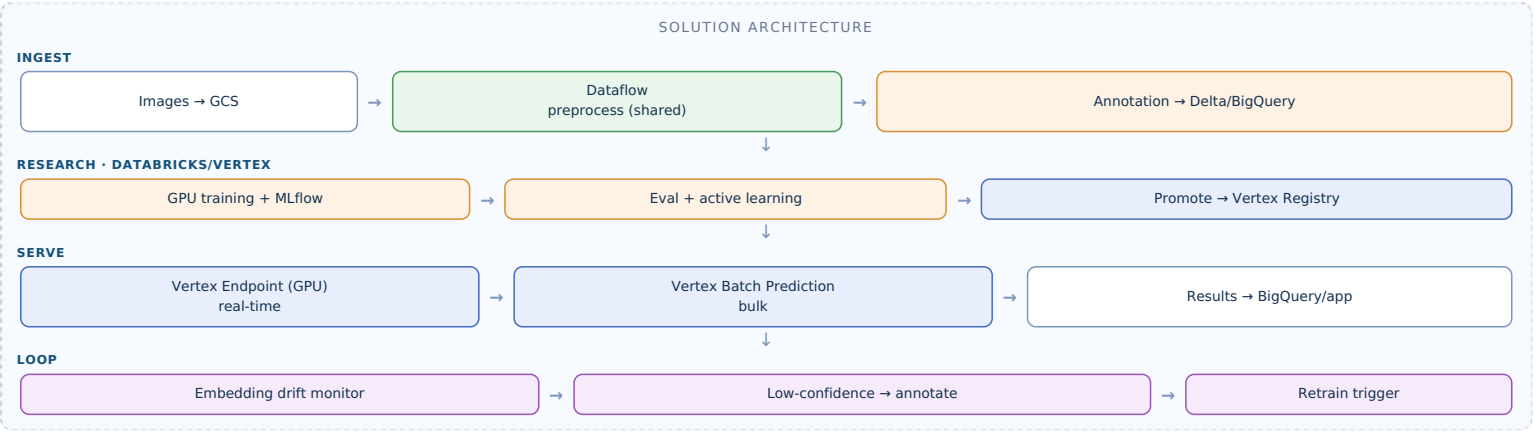
Step 5 — Common Mistake

Tuning for maximum sensitivity and drowning on-call in false positives until everyone mutes the alerts — then the one real incident is missed. The goal is **actionable** alerts: correlate, dedupe, require persistence, and incorporate on-call feedback. Also ignoring seasonality, which makes the system cry wolf on every predictable traffic peak.

ML QUESTION 13

Design a large-scale image classification / computer-vision pipeline

Focus: Vision at scale, GPU serving, batch + real-time, data pipeline



Step 1 — Clarify Requirements

Use case: tagging uploads, defect detection, medical — accuracy bar and latency differ hugely. **Volume:** images/sec, sizes. **Mode:** real-time at upload vs nightly batch over a corpus. **Hardware:** GPU/TPU budget. **Labels:** how is labeled data produced (annotation pipeline)? Confirm class imbalance, the need for active learning, and whether outputs feed humans or downstream systems.

Step 2 — High-Level Design

A data + serving pipeline. Images land in **GCS**; a preprocessing step (resize/normalize) runs in **Dataflow**. Training happens on **Databricks/Vertex** with GPUs over a labeled dataset (annotation tool → Delta/BigQuery). The model is registered and served on **Vertex Endpoints** (GPU) for real-time or **Vertex Batch Prediction** for bulk scoring. An **active-learning** loop sends low-confidence images to annotators. Monitoring tracks accuracy and data drift (new image distributions).

Step 3 — Deep Dive

GPU efficiency: dynamic batching and right-sized accelerators keep utilization high; batch mode for non-interactive scoring is far cheaper. **Preprocessing parity:** identical resize/normalize offline and online to avoid skew. **Active learning:** route uncertain/novel images to humans to maximize label value. **Drift:** image distributions shift (new cameras, lighting); monitor embeddings for drift and retrain. **Throughput:** for batch, parallelize across workers and write results to BigQuery/GCS.

Step 4 — Trade-offs to Discuss

- **Real-time vs batch serving:** batch maximizes GPU utilization and cost; real-time for user-facing upload tagging.
- **Bigger model vs latency/cost:** accuracy vs inference cost; distill or quantize for serving.
- **Label more vs active learning:** active learning gets more accuracy per labeling dollar.
- **On-device vs cloud inference:** on-device cuts latency/cost and aids privacy but limits model size.

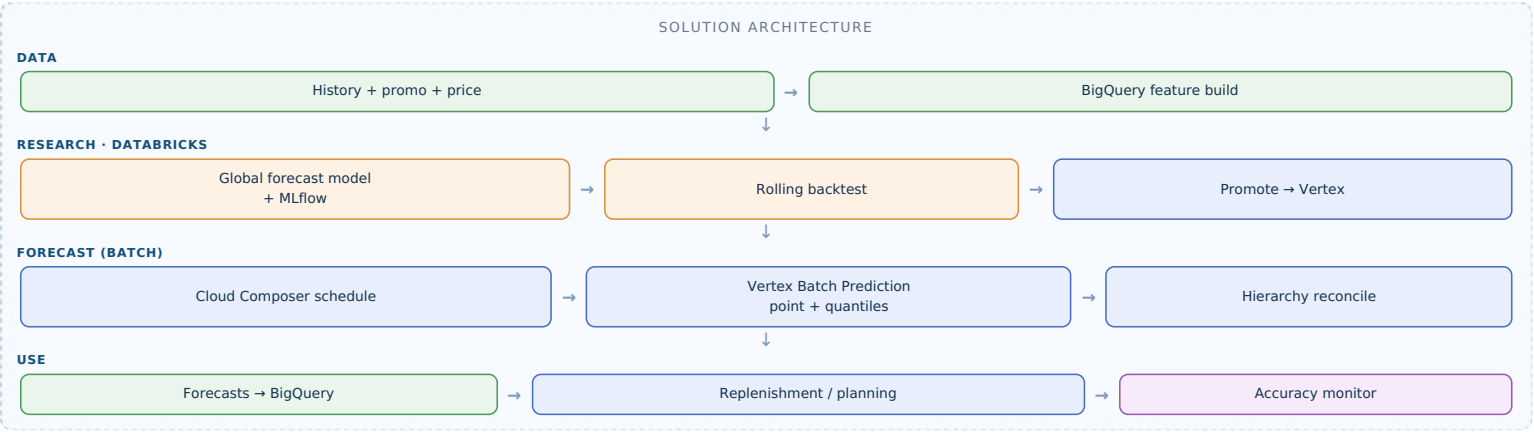
Step 5 — Common Mistake

Letting preprocessing differ between training and serving (different resize/normalization libs or parameters), which silently tanks production accuracy — a vision-specific flavor of training-serving skew. Pin the exact preprocessing in shared code. Also: ignoring drift from new image sources, and serving an oversized model real-time when batch + distillation would be far cheaper.

ML QUESTION 14

Design a demand forecasting system (retail / supply chain)

Focus: Time-series forecasting at scale, hierarchical, batch, business integration



Step 1 — Clarify Requirements

Granularity: per SKU x store x day? That can be millions of series. **Horizon:** days, weeks, months? **Cadence:** daily/weekly batch is typical. **Inputs:** history, promotions, price, weather, holidays. **Consumer:** inventory/replenishment planning — so we need prediction intervals, not just point estimates. Confirm we handle intermittent demand (many zeros), new products (cold start), and hierarchical consistency (store totals = sum of SKUs).

Step 2 — High-Level Design

A batch forecasting pipeline. Features (history, promo calendar, price, exogenous signals) assembled in **BigQuery**; models trained on **Databricks** (global model across series, or per-segment) and run via **Vertex Batch Prediction** on a schedule (Cloud Composer). Forecasts + intervals write to BigQuery and feed planning systems / Looker. A backtesting harness validates accuracy across horizons before promotion. New products use hierarchy-based or attribute-based cold-start forecasts.

Step 3 — Deep Dive

Global vs local models: a single global model trained across all series shares structure and handles sparse series better than thousands of per-series models. **Prediction intervals:** planning needs uncertainty (quantile forecasts) to set safety stock. **Hierarchy:** reconcile forecasts so SKU forecasts sum to category/store totals. **Intermittent demand:** special handling for many-zero series. **Backtesting:** rolling-origin evaluation across horizons; promote only if it beats the current baseline. **Exogenous signals:** promotions and holidays dominate — encode them carefully.

Step 4 — Trade-offs to Discuss

- **Global vs per-series models:** global scales and generalizes; per-series can fit idiosyncratic items but is unmaintainable at millions of series.
- **Point vs probabilistic forecast:** intervals cost more but are essential for inventory decisions.
- **Classical (ETS/ARIMA) vs ML:** classical is robust and explainable; ML captures cross-series and exogenous effects.
- **Frequency of retrain:** more frequent adapts faster but costs compute and risks instability.

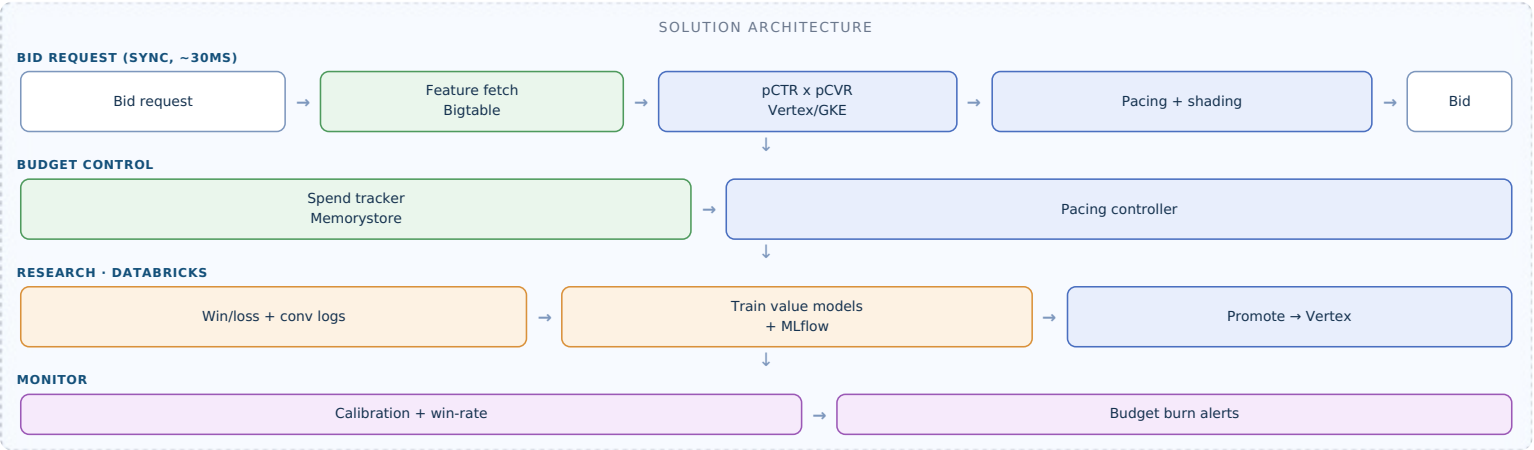
Step 5 — Common Mistake

Producing only point forecasts and evaluating on aggregate error, then watching inventory decisions fail because uncertainty was ignored and the model was tuned on overall MAPE while the high-value SKUs were poor. Provide prediction intervals, backtest per-segment and per-horizon, and reconcile the hierarchy. Also: training thousands of brittle per-series models instead of one robust global model.

ML QUESTION 15

Design a real-time bidding (RTB) system for programmatic ads

Focus: Ultra-low-latency, budget pacing, value estimation, auction



Step 1 — Clarify Requirements

Latency: brutal — full bid decision in ~10-50 ms or the auction is lost. **Scale:** hundreds of thousands of bid requests/sec. **Objective:** maximize value (conversions) within budget and pacing constraints. **Inputs:** pCTR/pCVR estimates, user value, campaign budgets. Confirm we need budget pacing (don't spend the day's budget in an hour) and bid shading for the auction type (first vs second price).

Step 2 — High-Level Design

A bidder service that, per request, fetches features, calls value-estimation models (pCTR x pCVR x value, served on **Vertex/GKE**), applies a bidding policy with pacing and shading, and returns a bid within the latency budget. Budgets and spend tracked in a fast store (**Bigtable/Memorystore**) updated by a pacing controller. Win/loss and conversion logs stream to BigQuery for model training on **Databricks**. Everything is horizontally sharded for QPS.

Step 3 — Deep Dive

Latency: precompute everything possible, fetch features in <5 ms, keep models tiny; co-locate to minimize network hops. **Pacing:** a control loop adjusts bid aggressiveness to spend budget smoothly across the day. **Bid shading:** in first-price auctions, bid below true value to avoid overpaying; learn the shading from win-rate curves. **Value estimation:** calibrated pCTR/pCVR feed the bid; calibration errors directly waste budget. **Feedback delay:** conversions arrive late, complicating attribution and training.

Step 4 — Trade-offs to Discuss

- **Bid accuracy vs latency:** a richer value model that misses the 30 ms window wins nothing.
- **Aggressive vs smooth pacing:** spend fast to capture value vs pace to last all day and avoid CPM spikes.
- **Shading sophistication vs robustness:** complex shading can overfit win-rate noise.
- **Centralized vs edge bidders:** edge cuts latency; central simplifies budget consistency.

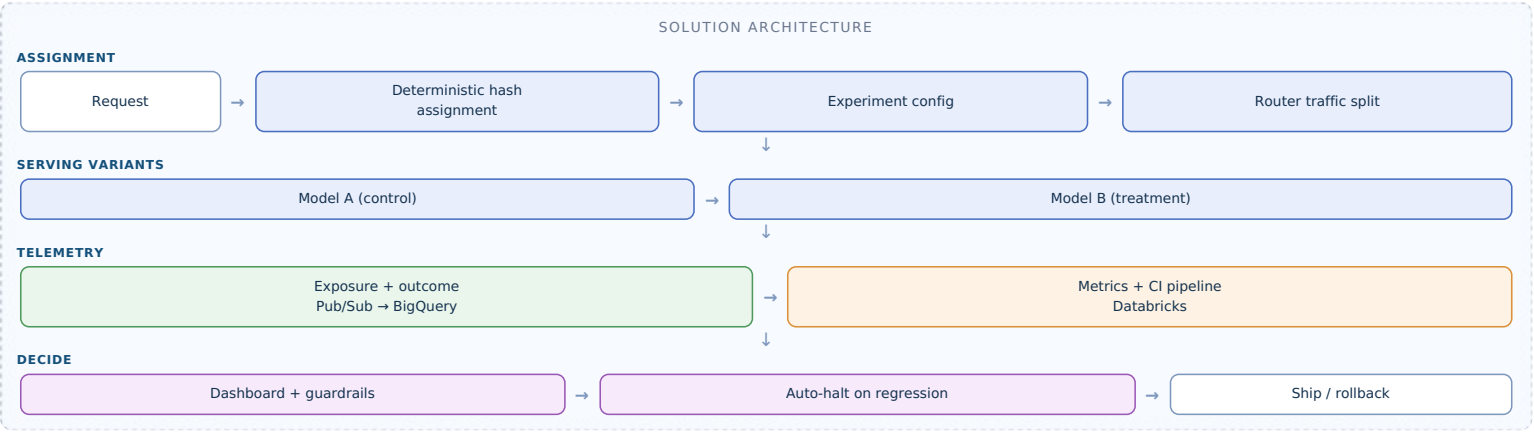
Step 5 — Common Mistake

Building a brilliant value model that can't meet the latency budget — in RTB, a slightly worse but fast decision beats a perfect-but-late one every time, because late bids are simply discarded. Engineer for the tail latency first. Also: ignoring pacing and burning the daily budget in the first hour, or mis-calibrated pCVR systematically overbidding.

ML QUESTION 16

Design an A/B testing & experimentation platform for ML models

Focus: Experimentation infra, online evaluation, statistical rigor, guardrails



Step 1 — Clarify Requirements

Who uses it: ML teams comparing model versions. **Needs:** consistent user assignment, metric pipelines, statistical significance, guardrail metrics, and ramp controls. **Scale:** many concurrent experiments. **Constraints:** avoid interference between experiments, ensure assignment is sticky and unbiased. Confirm we support shadow tests, canary ramps, and that we measure both the target metric and guardrails (latency, revenue, error rates) so a 'win' that breaks something is caught.

Step 2 — High-Level Design

An assignment service hashes user/request IDs into variants deterministically; a config service defines experiments and ramps. Model variants serve behind the **Vertex Endpoint** router with traffic splits. Exposure + outcome events log to **Pub/Sub → BigQuery**; a metrics pipeline computes treatment effects and confidence intervals (analysis often on **Databricks**). A dashboard shows results and guardrails; automated checks can stop an experiment that trips a guardrail.

Step 3 — Deep Dive

Assignment: deterministic hashing gives sticky, balanced, independent assignment; isolate overlapping experiments with layered randomization. **Statistics:** pre-register metrics and sample size, account for multiple comparisons and peeking; use CUPED/variance reduction for sensitivity. **Guardrails:** monitor latency, errors, and revenue alongside the target; auto-halt on regression. **Ramp:** shadow → small canary → ramp, with monitoring at each step. **Network effects:** for marketplaces, cluster-randomize to avoid interference.

Step 4 — Trade-offs to Discuss

- **Speed vs rigor:** peeking and early stops are tempting but inflate false positives; sequential tests help.
- **User vs cluster randomization:** cluster avoids interference but reduces power.
- **Shared platform vs bespoke tests:** a platform enforces rigor and reuse but is upfront work.
- **Online experiment vs offline replay:** offline is cheap but can mislead; online is truth but costly.

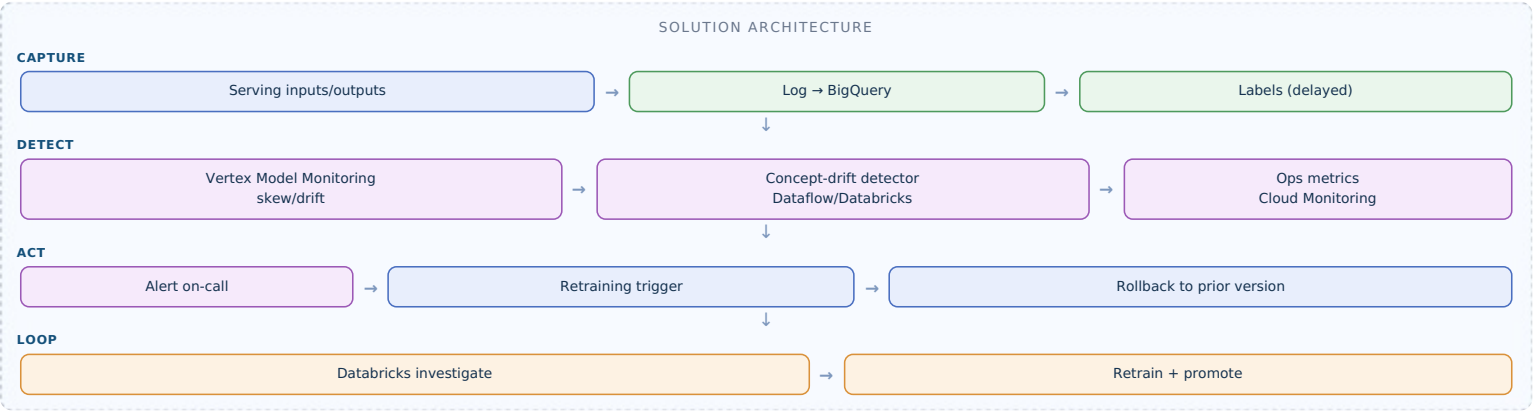
Step 5 — Common Mistake

Trusting offline metrics or peeking at running experiments and calling a winner early — both produce confident, wrong decisions. Pre-register metrics and sample size, use guardrails to catch collateral damage, and ramp with monitoring. Also: ignoring interference in marketplaces, where treating one side affects the control group and biases the result.

ML QUESTION 17

Design a model monitoring & drift detection system

Focus: Production observability, data/concept drift, skew, alerting, retraining loop



Step 1 — Clarify Requirements

What to monitor: input data quality, feature drift, prediction drift, concept drift (accuracy decay), and operational metrics (latency, errors). **Label availability:** immediate, delayed, or none? That dictates whether we can measure live accuracy or must rely on proxy/drift signals. **Scale:** many models. **Action:** alert, auto-retrain, or roll back. Confirm the SLO and who owns response.

Step 2 — High-Level Design

A monitoring layer that captures serving inputs/outputs (logged to BigQuery), computes reference-vs-live distribution comparisons, and tracks operational metrics in Cloud Monitoring. **Vertex Model Monitoring** handles feature skew/drift out of the box; custom detectors (on **Databricks/Dataflow**) cover concept drift once labels arrive. Alerts route to on-call; significant drift can trigger a retraining pipeline. A dashboard unifies data, model, and ops health per model version.

Step 3 — Deep Dive

Skew vs drift: skew = training-vs-serving mismatch (catch at launch); drift = serving distribution moving over time. **Delayed labels:** when ground truth lags, rely on input/prediction drift as early warning and confirm with accuracy when labels land. **Concept drift:** the input-output relationship changes even if inputs look stable — only labels reveal it. **Thresholds:** tune to avoid alert fatigue; use statistical distance with sensible bounds. **Loop:** connect alerts to the retraining trigger and feedback into Databricks.

Step 4 — Trade-offs to Discuss

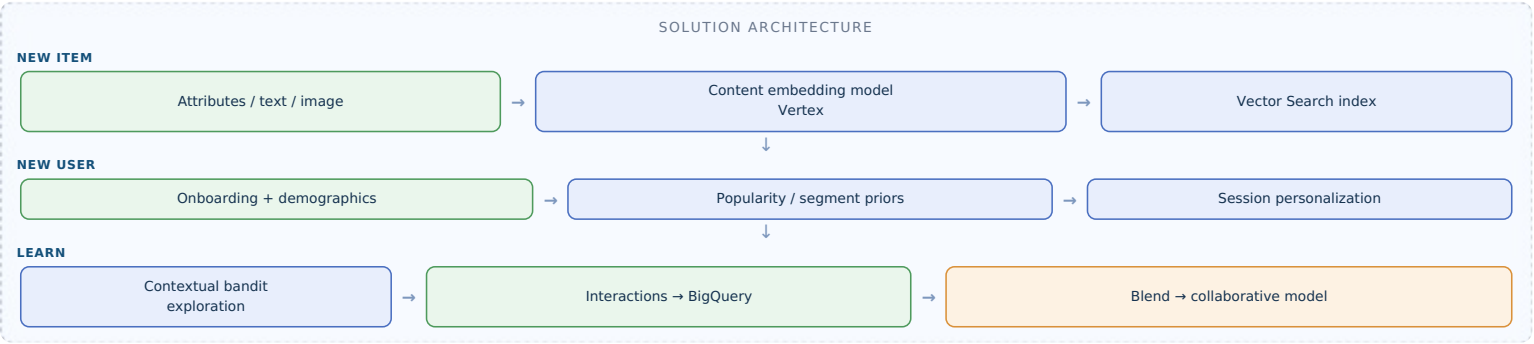
- **Proxy drift vs true accuracy:** drift is available immediately but is only a proxy; accuracy is truth but delayed.
- **Sensitivity vs noise:** tight thresholds catch issues early but cry wolf.
- **Auto-retrain vs human review:** automation reacts fast but can retrain on bad data; gates add safety.
- **Per-feature vs holistic monitoring:** granular pinpoints causes; holistic is cheaper.

Step 5 — Common Mistake

Monitoring only system health (latency, errors) and assuming the model is fine because the service is up — the model can silently rot from drift while every infra metric stays green. Monitor data and prediction distributions and, once labels arrive, accuracy. Also: alerting on drift with no defined response, so alerts pile up and nobody retrains.

Design a cold-start solution for a recommendation system

Focus: New users/items, content features, exploration, bootstrapping



Step 1 — Clarify Requirements

Which cold start: new users (no history), new items (no interactions), or a brand-new system? Each needs a different tactic. **Goal:** deliver useful recs immediately and gather signal fast to warm up. **Constraints:** latency, catalog churn rate (e.g. news vs durable goods). Confirm we can collect side information (item attributes, user profile/onboarding) and that some exploration is acceptable to learn.

Step 2 — High-Level Design

A hybrid that degrades gracefully. **New items:** represent with content embeddings (text/image attributes via a model) so they're retrievable in **Vector Search** before any interaction. **New users:** use onboarding signals, demographics, and popularity priors; quickly personalize as interactions arrive. **Exploration:** a bandit layer surfaces uncertain items to gather feedback. As interaction data accrues, blend toward the collaborative model. Trained on **Databricks**, served on **Vertex**.

Step 3 — Deep Dive

Content-based bootstrap: attribute/content embeddings let new items enter retrieval immediately; this is the core fix for item cold start. **Popularity + context priors:** for new users, start with trending/ segment-popular items, then personalize from session behavior within minutes. **Exploration:** contextual bandits balance showing safe popular items vs exploring uncertain ones to learn faster; without exploration, new items never get the impressions they need. **Blending:** weight collaborative vs content signal by how much interaction data exists.

Step 4 — Trade-offs to Discuss

- **Exploration vs exploitation:** exploration warms up the catalog but costs short-term engagement.
- **Content vs collaborative:** content handles cold start; collaborative is stronger once data exists — blend by data availability.
- **Onboarding friction vs signal:** asking new users for preferences helps but adds friction.
- **Global popularity vs segment popularity:** segment priors personalize cold start at some complexity.

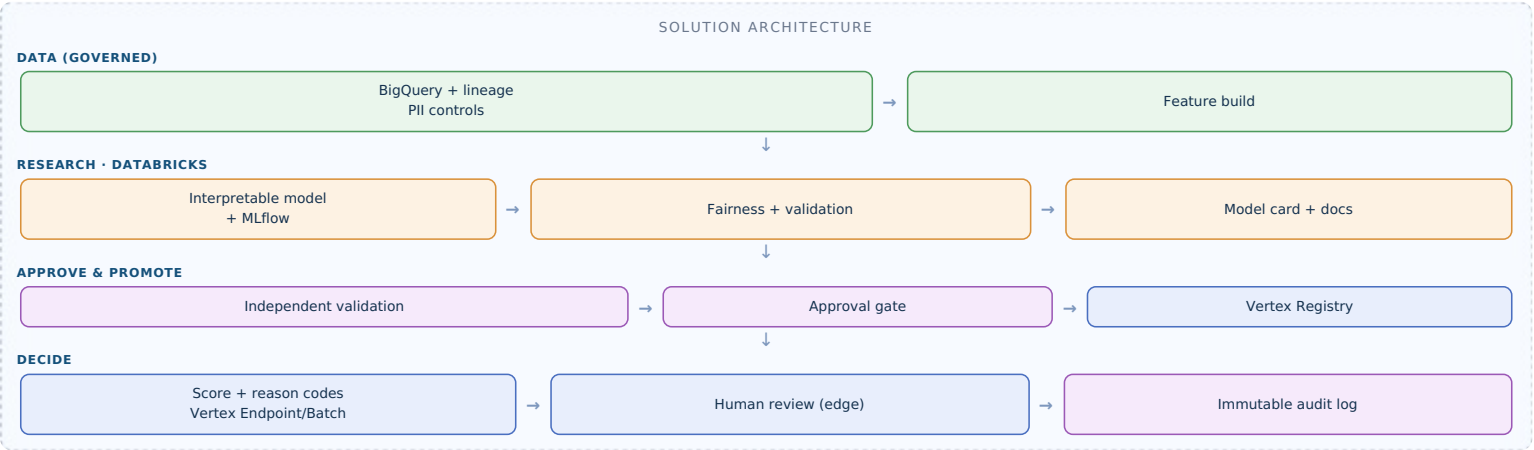
Step 5 — Common Mistake

Relying purely on collaborative filtering, so new items get zero impressions (no interactions → never recommended → never get interactions) and new users see generic junk. The fix is content-based representations plus deliberate exploration. Also: never transitioning out of cold-start mode — you must blend toward the personalized model as data accumulates.

ML QUESTION 19

Design a credit scoring / loan risk model system

Focus: Regulated ML, explainability, fairness, auditability, batch + real-time



Step 1 — Clarify Requirements

Regulatory context: credit decisions require explainability (adverse-action reasons), fairness/non-discrimination, and full auditability — this dominates the design. **Decision mode:** real-time at application or batch. **Data:** credit history, income, behavior; sensitive attributes must be handled carefully. **Stability:** models must be monitored and re-validated. Confirm governance: model risk management, documentation, and human override.

Step 2 — High-Level Design

A governed scoring pipeline emphasizing transparency. Features from **BigQuery** (with strict lineage/PII controls); models trained on **Databricks** with documented data and validation, promoted through a heavy approval gate to **Vertex**. Scoring (real-time endpoint or batch) returns a score plus **reason codes**. A decisioning layer applies policy and human review for edge cases. Every decision, model version, and input is logged immutably for audit and appeals.

Step 3 — Deep Dive

Explainability: use inherently interpretable models or robust post-hoc explanations to generate legally required adverse-action reasons. **Fairness:** test for disparate impact across protected groups and document mitigations; you cannot simply drop sensitive attributes (proxies remain). **Auditability:** immutable logging of inputs, model version, score, and decision. **Governance:** independent model validation, documented model cards, and re-validation cadence. **Stability:** monitor population shift; changes require re-approval, not silent auto-deploy.

Step 4 — Trade-offs to Discuss

- **Accuracy vs explainability:** regulated lending often favors interpretable models even at some accuracy cost.
- **Auto-decision vs human review:** automation scales; humans handle edge cases and provide accountability.
- **More data vs fairness/privacy risk:** richer features can encode bias and raise compliance risk.
- **Fast iteration vs governance:** regulated models can't auto-promote; gates slow but protect.

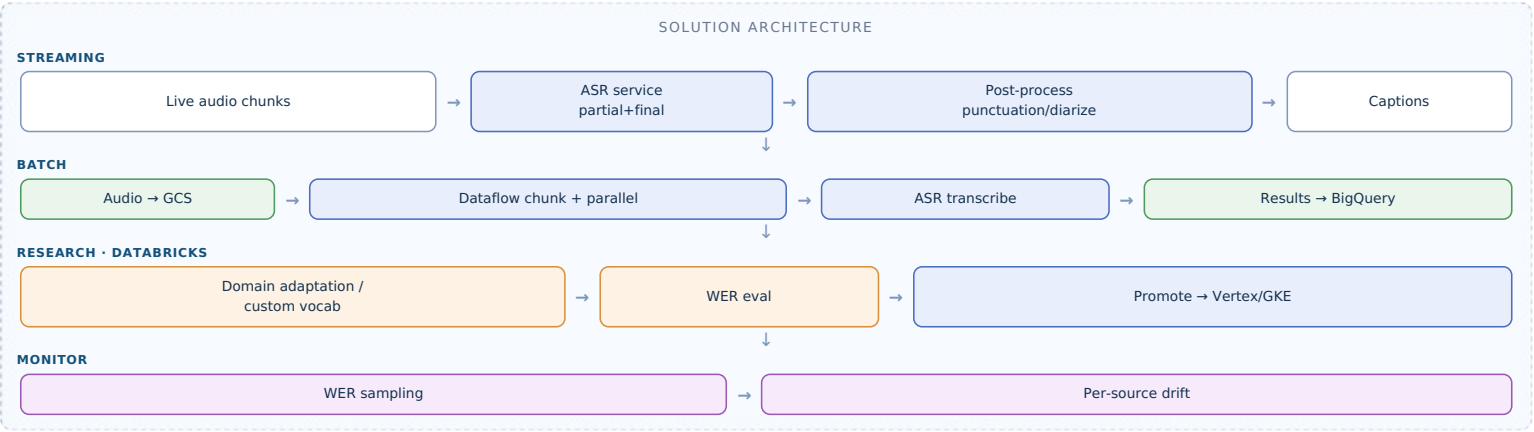
Step 5 — Common Mistake

Treating it like an unregulated optimization problem — chasing AUC with an opaque model, no reason codes, no fairness testing, and silent auto-deploys. That's non-compliant and legally dangerous. Explainability, fairness auditing, immutable audit trails, and human-gated promotion are requirements, not nice-to-haves. Also: dropping protected attributes and assuming the model is now 'fair' while proxies persist.

ML QUESTION 20

Design a production speech-to-text (ASR) pipeline

Focus: Audio ML, streaming vs batch, latency, post-processing, domain adaptation



Step 1 — Clarify Requirements

Mode: streaming (live captions, <300 ms partial results) vs batch (transcribe recordings)? **Domain:** general vs specialized vocabulary (medical, legal) needing adaptation. **Languages & accents.** **Quality bar:** word error rate target. **Volume:** hours of audio/day. Confirm whether we use a managed API (**Vertex / Speech-to-Text**) or self-host a model for control/cost, and whether we need diarization (who spoke) and punctuation.

Step 2 — High-Level Design

Two paths. **Streaming:** audio chunks stream to a low-latency ASR service returning partial+final transcripts, with light post-processing (punctuation, formatting). **Batch:** files in **GCS** → Dataflow orchestrates chunking and parallel transcription → results to BigQuery/GCS. Use Google **Speech-to-Text** or a self-hosted model on **Vertex/GKE (GPU)**. Domain adaptation (custom vocab/fine-tune) is developed on **Databricks**. Post-processing handles diarization, punctuation, and redaction.

Step 3 — Deep Dive

Streaming latency: chunked inference with partial hypotheses; trade a little accuracy for responsiveness. **Domain adaptation:** custom vocabularies or fine-tuning sharply cut WER on jargon; manage as a research-to-prod artifact. **Post-processing:** inverse text normalization, punctuation, diarization, and PII redaction are a pipeline after the acoustic model. **Scale (batch):** chunk long audio and parallelize; handle retries. **Quality monitoring:** sample transcripts for WER on a labeled set; watch drift by audio source.

Step 4 — Trade-offs to Discuss

- **Managed API vs self-host:** managed is fast to ship and robust; self-host gives cost control, privacy, and customization.
- **Streaming vs batch:** streaming for live UX, batch for cheaper, higher-accuracy offline.
- **Latency vs accuracy:** smaller chunks = lower latency but more errors at boundaries.
- **General vs domain model:** domain adaptation lifts accuracy but adds maintenance.

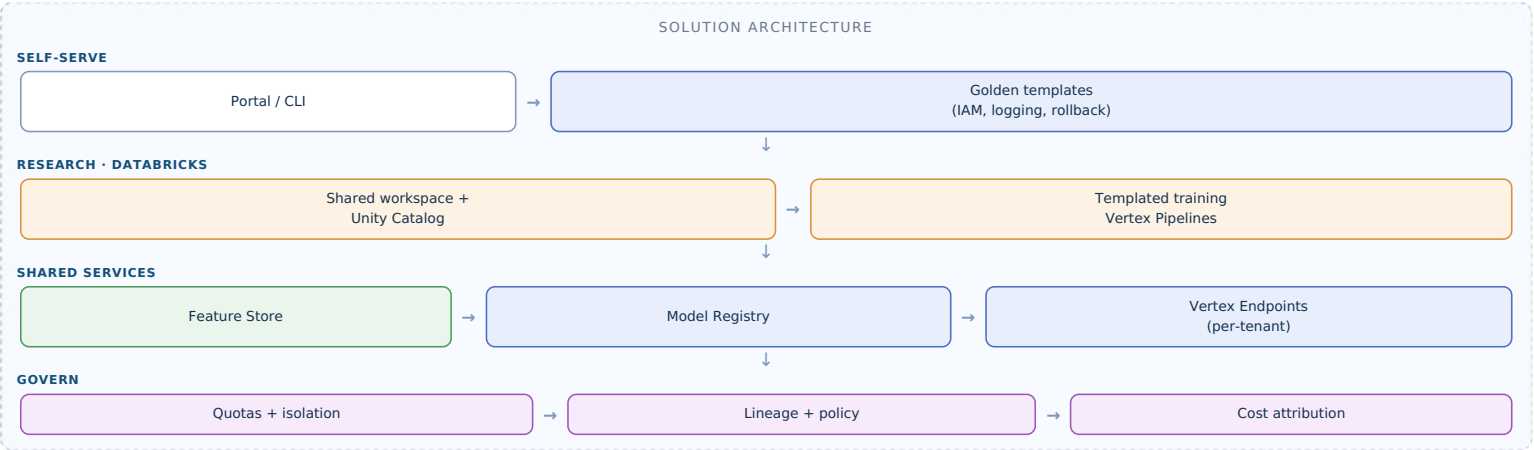
Step 5 — Common Mistake

Skipping the post-processing and domain-adaptation layers and judging the system on raw acoustic output — users experience the formatted transcript (punctuation, casing, jargon), and a generic model on specialized audio produces a high WER that no amount of infra fixes. Plan for custom vocab/fine-tuning and a proper post-processing stage, and monitor WER per audio source.

ML QUESTION 21

Design a multi-tenant internal ML platform (self-serve for many teams)

Focus: Platform engineering, paved paths, isolation, governance, cost attribution



Step 1 — Clarify Requirements

Users: many ML teams of varying maturity wanting to train, deploy, and monitor models self-serve. **Goals:** reduce time-to-production, enforce standards (security, monitoring, lineage), and attribute cost. **Isolation:** tenant data and workloads must not interfere. **Flexibility vs guardrails:** how much can teams customize? Confirm the platform spans data → features → training → serving → monitoring as a 'paved path.'

Step 2 — High-Level Design

A paved-path platform over GCP primitives. Shared **feature store**, a templated **Vertex Pipelines** training path, a central **model registry**, standardized **Vertex Endpoint** deployment with built-in monitoring, and a self-serve portal/CLI. **Databricks** is the shared research workspace with Unity Catalog governance. Multi-tenancy via projects/namespaces, quotas, and IAM. Golden templates encode security, logging, and rollback so teams get production-grade defaults for free.

Step 3 — Deep Dive

Paved path vs escape hatch: 80% of teams use templates; advanced teams can drop down to raw primitives without leaving the platform. **Isolation:** per-tenant projects/namespaces, network policies, and quotas prevent noisy neighbors and data leakage. **Governance:** central lineage, access control, and policy enforcement (no model ships without monitoring). **Cost attribution:** labels/billing export per tenant to drive accountability. **Self-serve:** portal/CLI for one-command deploy with sane defaults; golden templates keep standards high.

Step 4 — Trade-offs to Discuss

- **Standardization vs flexibility:** opinionated paths speed delivery but frustrate advanced teams — provide escape hatches.
- **Central vs federated ownership:** central enforces standards; federated scales but risks drift.
- **Build vs adopt managed:** lean on Vertex managed services to avoid reinventing serving/monitoring.
- **Strict isolation vs resource efficiency:** hard isolation is safe but underutilizes shared GPUs.

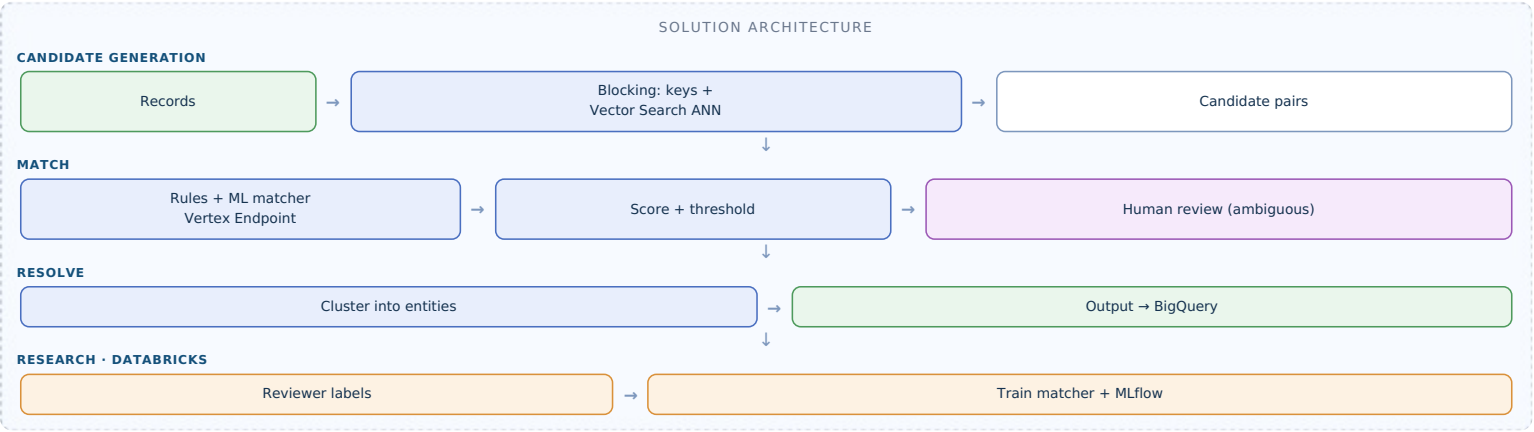
Step 5 — Common Mistake

Building a rigid platform that only fits the average use case, so advanced teams route around it and the platform fragments — or the opposite, a free-for-all with no guardrails where every team reinvents monitoring and security badly. The answer is a paved path with escape hatches: great defaults, enforced standards (monitoring, lineage), and room to customize. Also: ignoring cost attribution until the bill explodes.

ML QUESTION 22

Design a near-duplicate detection / entity resolution system

Focus: Similarity at scale, blocking, embeddings, precision/recall trade-off



Step 1 — Clarify Requirements

Domain: dedupe products, customer records, documents? **Scale:** matching N records is $O(N^2)$ naively — clarify size (millions+) to justify blocking. **Match definition:** exact, fuzzy, semantic? **Mode:** batch dedupe vs real-time 'is this new record a duplicate?' **Cost asymmetry:** false merges (combining distinct entities) are often worse than false splits. Confirm whether human review resolves ambiguous pairs.

Step 2 — High-Level Design

A blocking + matching pipeline to avoid all-pairs comparison. **Blocking/candidate generation** groups plausibly-matching records (via keys or embedding ANN in **Vector Search**); a **pairwise matcher** (rules + ML, trained on **Databricks**) scores candidate pairs; a **clustering/resolution** step groups matches into entities. Batch runs in **Dataflow/BigQuery**; real-time checks query the index. Ambiguous pairs route to human review whose decisions become labels.

Step 3 — Deep Dive

Blocking is the scalability key: reduce $O(N^2)$ to manageable candidate pairs via keys or embedding ANN; recall of blocking caps overall recall. **Matching:** combine deterministic rules (exact IDs) with a learned model over similarity features (string distance, embeddings). **Resolution:** transitive clustering with care — bad merges cascade. **Threshold:** tune precision/recall to the cost asymmetry; route the uncertain band to humans. **Incremental:** for real-time, match a new record against the existing index without reprocessing everything.

Step 4 — Trade-offs to Discuss

- **Blocking recall vs cost:** looser blocking finds more matches but compares more pairs.
- **Precision vs recall:** favor precision when false merges are costly; favor recall when missing dupes is worse.
- **Rules vs ML matcher:** rules are transparent and cheap; ML handles fuzzy cases.
- **Batch vs incremental:** batch is simpler; incremental is needed for real-time intake.

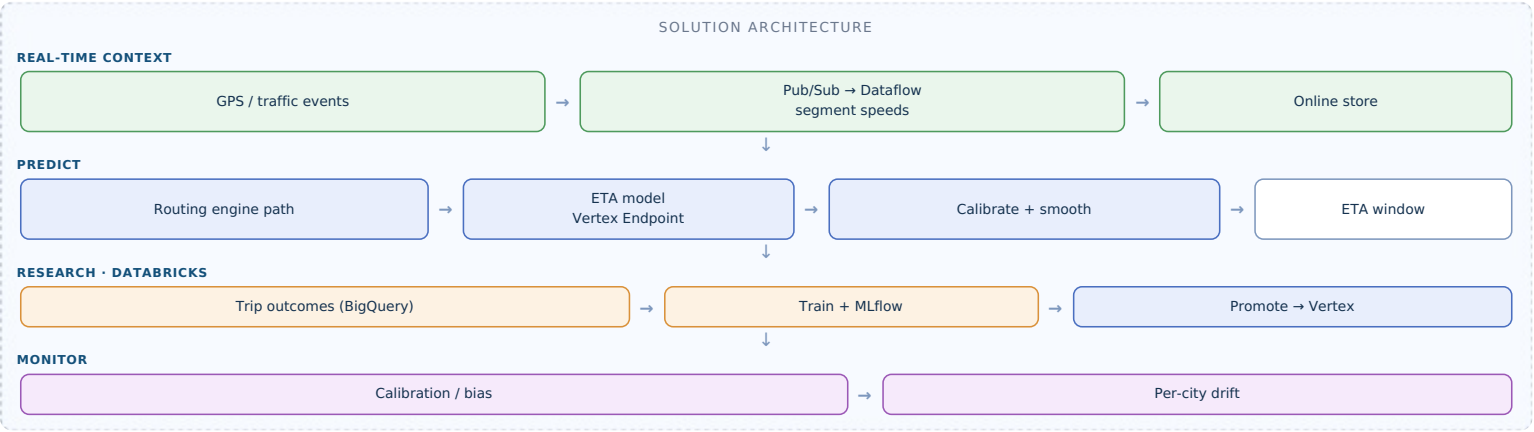
Step 5 — Common Mistake

Skipping blocking and attempting all-pairs comparison, which is computationally impossible at scale, or setting blocking too tight so true matches never become candidates (capping recall regardless of the matcher). Design blocking deliberately and measure its recall. Also: aggressive transitive merging that chains unrelated records into one giant wrong entity.

ML QUESTION 23

Design an ETA / arrival-time prediction system (ride-share / delivery)

Focus: Geospatial ML, real-time features, routing integration, continuous correction



Step 1 — Clarify Requirements

What ETA: driver-to-pickup, trip duration, delivery window? **Latency:** real-time, <100 ms. **Inputs:** route, live traffic, historical segment speeds, weather, time-of-day. **Accuracy target:** and how it's measured (calibration of the time window matters for UX). **Update cadence:** ETAs should refresh during the trip. Confirm integration with the routing engine and that we need uncertainty (range, not just a point).

Step 2 — High-Level Design

A real-time prediction service on top of routing. The routing engine proposes a path; an ML model (trained on **Databricks**, served on **Vertex**) predicts travel time using segment-level features, live traffic, and context from a fast online store fed by **Pub/Sub -> Dataflow**. ETAs refresh as the trip progresses. Trip outcomes (actual durations) log to BigQuery to retrain. Historical segment speed aggregates are precomputed.

Step 3 — Deep Dive

Hybrid physics + ML: use the routing engine's distance/segments as structure, and ML to correct for real-world delays (traffic, intersections, pickup time). **Real-time traffic:** live segment speeds in a low-latency store dominate accuracy in cities. **Continuous correction:** re-predict as position updates; large mid-trip jumps hurt trust, so smooth. **Uncertainty:** provide a window; calibrate so 'arrives in 10 min' is right ~the claimed fraction of the time. **Geospatial features:** segment embeddings, time-of-day, weather.

Step 4 — Trade-offs to Discuss

- **Point vs range ETA:** a range is honest and calibratable but UX often wants a single number — blend.
- **Pure ML vs routing+ML hybrid:** hybrid leverages map structure and generalizes better.
- **Refresh frequency vs stability:** frequent updates are accurate but jumpy ETAs erode trust.
- **Live traffic cost vs accuracy:** real-time speeds are expensive to maintain but essential in dense areas.

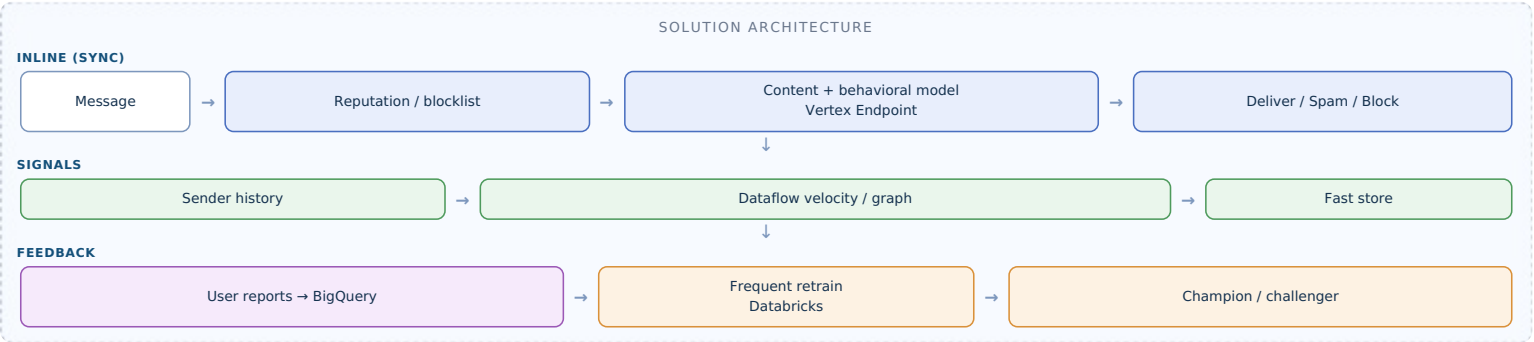
Step 5 — Common Mistake

Optimizing average error while shipping a poorly **calibrated** ETA — users care that the promised time is trustworthy, and a low-MAE-but-biased ETA (always optimistic) destroys trust. Calibrate and report honest windows. Also: ignoring live traffic and relying on historical averages, which fails exactly when it matters (incidents, rush hour).

ML QUESTION 24

Design a spam / abuse detection system for messaging or email

Focus: Adversarial classification, fast model updates, precision-critical, feedback signals



Step 1 — Clarify Requirements

Channel: email, in-app messages, comments? **Latency:** inline at send/receive (<100 ms). **Cost asymmetry:** blocking legitimate messages (FP) is very costly to trust, while letting spam through (FN) is annoying but recoverable — usually precision-leaning. **Adversarial:** spammers adapt within hours, so fast retraining and feedback (user 'report spam') are essential. Confirm signals available: content, sender reputation, network/behavioral.

Step 2 — High-Level Design

A layered classifier. Fast reputation/blocklist checks first; then a content+behavioral model (trained on **Databricks**, served on **Vertex**) scores the message using text features, sender history, and network signals. A decision layer routes: deliver, spam-folder, or block. User reports ('this is spam'/'not spam') stream to BigQuery as fresh labels enabling frequent retrains. Sender reputation maintained in a fast store via streaming aggregates.

Step 3 — Deep Dive

Adversarial adaptation: spammers probe and morph; the system needs frequent retraining, champion-challenger, and signals beyond content (sender reputation, sending velocity, network graph) that are harder to fake. **Feedback loop:** user reports are gold-standard labels but noisy and delayed; weight them and guard against report-bombing. **Precision focus:** protect legitimate mail with conservative thresholds and a quarantine (spam folder) rather than hard block. **Reputation:** behavioral/graph features catch new spam campaigns that content filters miss.

Step 4 — Trade-offs to Discuss

- **Precision vs recall:** lean precision to protect legitimate messages; use quarantine for the uncertain band.
- **Content vs behavioral signals:** content is easy to spoof; behavioral/reputation is robust but needs infra.
- **Fast online updates vs stability:** rapid retrains track adversaries but risk instability.
- **Hard block vs spam folder:** folder is safer for FPs; block stops the worst abuse.

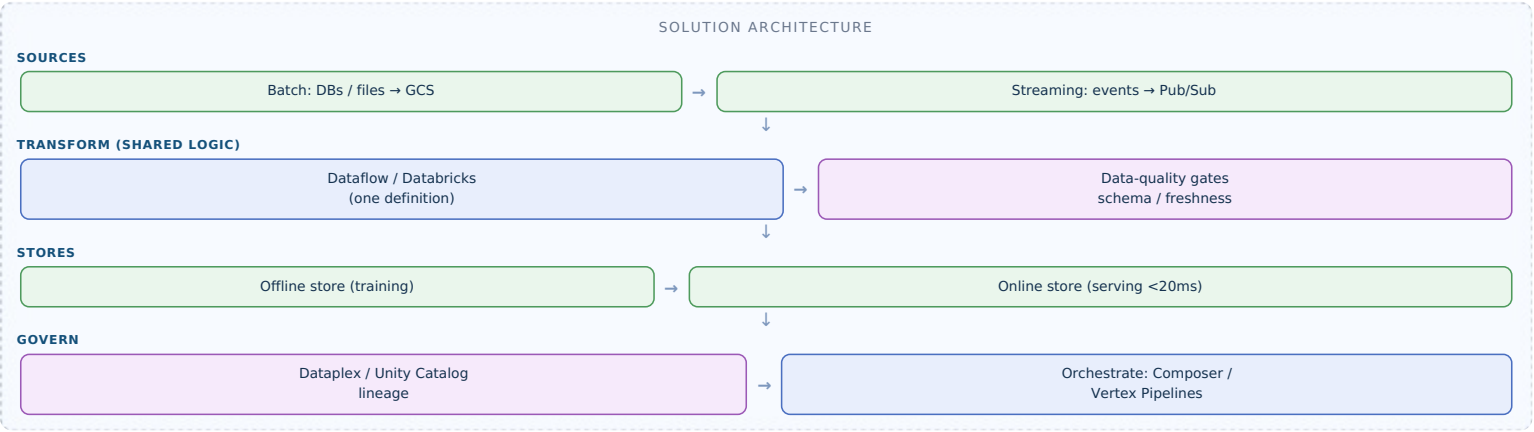
Step 5 — Common Mistake

Building a static content-only classifier and a fixed threshold, then watching spammers route around it within days while legitimate users get false-positived into oblivion. Spam is adversarial: combine content with hard-to-fake reputation/behavioral signals, retrain frequently with user-report feedback, and prefer quarantine over hard blocks to protect trust.

ML QUESTION 25

Design a feature engineering & data pipeline for ML (batch + streaming)

Focus: Data engineering for ML, data quality, lineage, batch/stream unification



Step 1 — Clarify Requirements

Sources: databases, events, logs, third-party? **Freshness tiers:** which features need streaming (seconds) vs batch (daily)? **Consumers:** training (large historical) and serving (low-latency lookup). **Quality bar:** schema enforcement, validation, handling late/missing data. **Scale:** volume and number of features. Confirm the non-negotiables: offline/online parity, lineage, and data-quality gates before features reach models.

Step 2 — High-Level Design

A unified pipeline feeding a feature store. Batch sources land in **GCS/BigQuery** and transform via **Dataflow/Dataproc** or **Databricks**; streaming sources flow **Pub/Sub → Dataflow**. A shared transformation library guarantees identical logic for both, writing to the **offline store** (training) and **online store** (serving). Data-quality checks (schema, ranges, freshness) gate the pipeline. Lineage and catalog via **Dataplex / Unity Catalog**. Orchestrated by Cloud Composer / Vertex Pipelines.

Step 3 — Deep Dive

Batch/stream unification: use one transformation definition (e.g. Beam) so a feature computes identically in both modes — the foundation of parity. **Data quality:** validate schema, distributions, nulls, and freshness at ingestion; quarantine bad data rather than poisoning models. **Late/missing data:** watermarks and backfill handle stragglers. **Lineage:** trace every feature to source and transform for debugging and governance. **Idempotency & backfill:** pipelines must be re-runnable to recompute history safely.

Step 4 — Trade-offs to Discuss

- **Streaming vs batch per feature:** stream only what needs freshness — streaming is the costly, complex part.
- **Lambda (separate batch+stream) vs Kappa (unified):** unified avoids divergent code paths and skew.
- **Strict quality gates vs availability:** strict gates protect models but can block pipelines; balance with quarantine.
- **Managed (Dataflow) vs Spark (Databricks):** managed for serving pipelines; Spark where research logic lives.

Step 5 — Common Mistake

Writing two separate codebases for batch (training) and streaming (serving) features, which inevitably diverge and cause training-serving skew — the model trains on one definition and serves on another. Unify the transformation logic. The second classic mistake: no data-quality gates, so a malformed upstream change silently corrupts features and degrades every model downstream with no alarm.

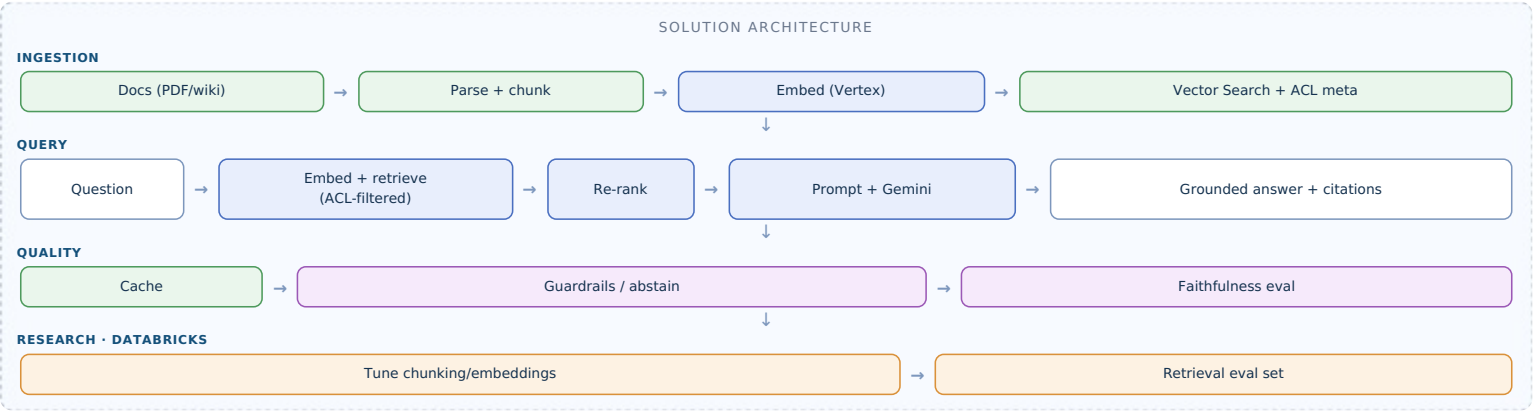
Section B · AI Agent / LLM / RAG System Design — 20 Questions

Generative-AI architecture on Google Cloud (Vertex AI, Gemini, Vector Search, Agent Builder). Same 5-step framework, with explicit research-to-production publishing paths.

AGENT QUESTION 1

Design a RAG system for enterprise document question-answering

Focus: Retrieval-augmented generation, chunking, vector search, grounding, citations



Step 1 — Clarify Requirements

Corpus: size, formats (PDF, wiki, tickets), update rate, access controls (per-user permissions on documents). **Latency:** first token <1 s, full answer 2–6 s. **Quality bar:** answers must be grounded with citations; hallucination is the main risk. **Scale:** users and QPS. Confirm we need permission-aware retrieval (a user must not see content they can't access) and freshness (new docs searchable quickly).

Step 2 — High-Level Design

An ingestion pipeline and a query pipeline. **Ingestion:** documents → parse → chunk → embed (Vertex embeddings) → index in **Vertex Vector Search** with metadata (ACLs, source). **Query:** embed the question, retrieve top-K permitted chunks, optionally re-rank, assemble a prompt with context, call **Gemini** on Vertex, and return a grounded answer with citations. Caching, guardrails, and feedback logging round it out. Embedding/chunking strategy is tuned in **Databricks**.

Step 3 — Deep Dive

Chunking: chunk size and overlap drive retrieval quality — too big dilutes relevance, too small loses context; structure-aware chunking helps. **Permission-aware retrieval:** filter by ACL metadata at query time so users only retrieve permitted content. **Grounding & citations:** instruct the model to answer only from context and cite sources; abstain when context is insufficient. **Re-ranking:** a cross-encoder re-ranks top candidates for precision. **Freshness:** incremental indexing on document change. **Eval:** measure retrieval recall and answer faithfulness.

Step 4 — Trade-offs to Discuss

- **Chunk size:** larger context vs sharper relevance — tune empirically.
- **Retrieve-K vs context cost:** more chunks improve recall but cost tokens/latency and can confuse the model.
- **Re-ranker on/off:** improves precision at extra latency/cost.
- **Managed RAG Engine vs custom:** managed is fast to ship; custom gives control over chunking, re-ranking, and ACLs.

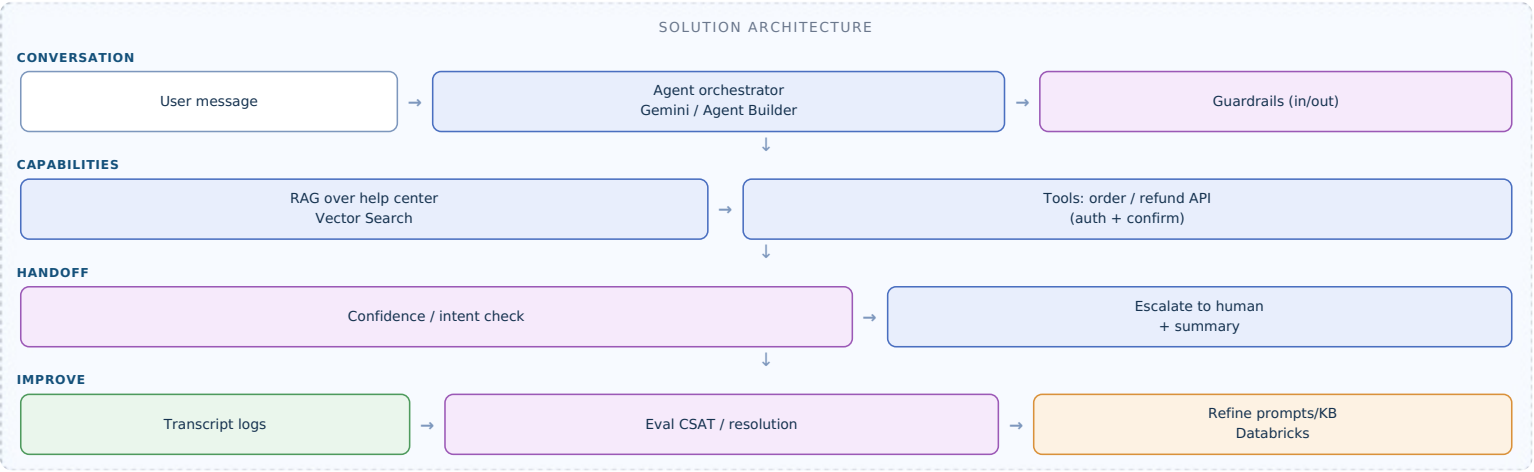
Step 5 — Common Mistake

Treating RAG as 'dump everything into the prompt' — poor chunking and no re-ranking retrieve irrelevant context, the model hallucinates or answers from the wrong doc, and there are no citations to verify. Worse: ignoring permissions so retrieval leaks restricted content into answers. Invest in chunking + retrieval quality, enforce ACLs at retrieval, require citations, and let the model abstain when unsure.

AGENT QUESTION 2

Design a customer-support AI agent (deflection + escalation)

Focus: Conversational agent, RAG + tools, escalation, safety, human handoff



Step 1 — Clarify Requirements

Scope: answer FAQs, take actions (refunds, order status), or both? **Channels:** chat, email, voice. **Guardrails:** what can it do autonomously vs require human approval (e.g. refunds over a threshold)? **KPIs:** deflection rate, CSAT, resolution time. **Escalation:** when and how to hand to a human with context. Confirm it must ground answers in the knowledge base and never invent policies, and that sensitive actions need authorization.

Step 2 — High-Level Design

A conversational orchestrator (Vertex Agent Builder / custom on Gemini) that combines **RAG** over the help center with **tool calls** to backend systems (order lookup, refund API). It maintains conversation state, applies guardrails, and escalates to a human with full context when confidence is low or policy requires. Knowledge base indexed in **Vector Search**; tools exposed via a function-calling layer with auth. Transcripts log for evaluation and improvement.

Step 3 — Deep Dive

Grounding vs action: separate 'answer from KB' (RAG) from 'do something' (authorized tool call); actions need permission checks and confirmation. **Escalation:** detect low confidence, user frustration, or out-of-scope intent and hand off with a summary so the human doesn't restart. **Guardrails:** input/output filters, refusal of unsafe requests, and limits on autonomous high-impact actions. **State:** track context across turns. **Eval:** measure resolution and CSAT, and review transcripts to find gaps; never claim policies not in the KB.

Step 4 — Trade-offs to Discuss

- **Autonomy vs safety:** more autonomous actions deflect more tickets but risk costly errors; gate high-impact actions.
- **Single agent vs RAG-only bot:** tool-using agents resolve more but are harder to make reliable.
- **Aggressive vs conservative escalation:** early escalation protects CSAT but lowers deflection.
- **Managed (Agent Builder) vs custom orchestration:** managed is faster; custom gives control.

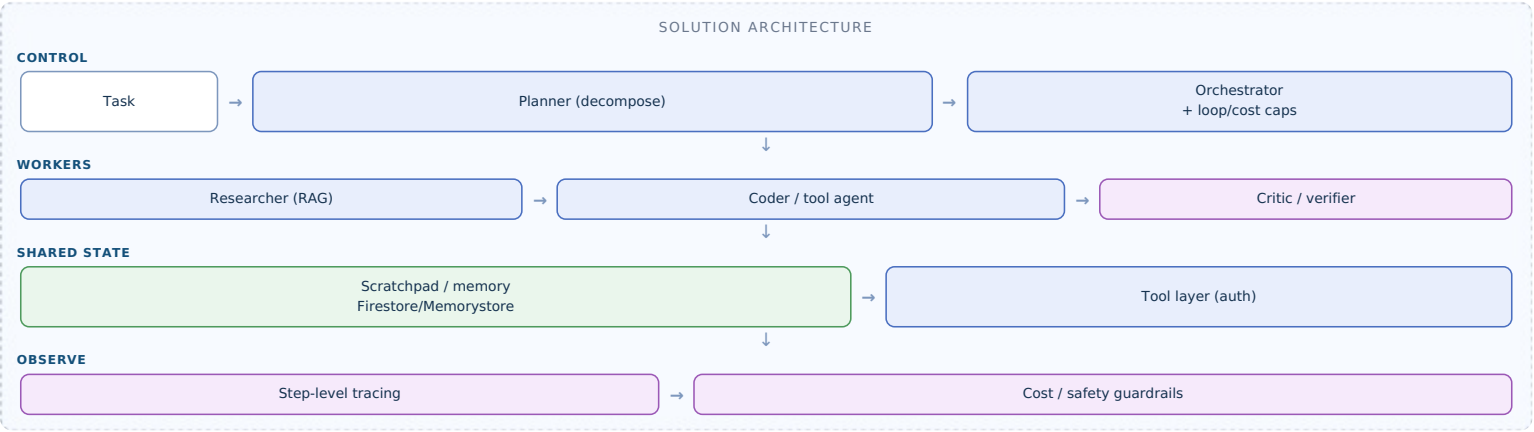
Step 5 — Common Mistake

Letting the agent confidently invent policies or take irreversible actions (issuing refunds) without grounding or authorization — a single hallucinated 'our policy allows...' can cost money and trust. Ground all factual answers in the KB, require auth + confirmation for impactful actions, and escalate gracefully with context rather than dead-ending the customer.

AGENT QUESTION 3

Design a multi-agent orchestration system for complex tasks

Focus: Agent orchestration, planning, delegation, state, reliability



Step 1 — Clarify Requirements

Task type: research, code, workflow automation? **Why multi-agent:** does the task truly need specialized roles (planner, researcher, coder, critic) or would one agent with tools suffice? **Control:** how do we bound cost, loops, and runaway behavior? **State:** shared memory/scratchpad across agents. **Reliability:** handling failures and verifying outputs. Confirm latency tolerance (these are often slower) and the autonomy/safety envelope.

Step 2 — High-Level Design

An orchestrator coordinates specialized agents (each a Gemini call with a role + tools). A **planner** decomposes the task into steps; **worker** agents execute (retrieval, code, API calls); a **critic/verifier** checks outputs; the orchestrator manages shared state and loop control. Built on Vertex (Gemini + function calling) with a state store (Firestore/Memorystore) and a tool layer. Guardrails cap iterations, cost, and tool scope. Traces log every step for debugging.

Step 3 — Deep Dive

Orchestration pattern: a controller (deterministic or LLM-driven) routes between agents; avoid fully free-form agent-to-agent chatter, which is hard to control. **State/memory:** a shared scratchpad and results store let agents build on each other. **Loop control:** hard caps on steps, cost, and time prevent runaway loops; a verifier reduces error propagation. **Reliability:** retries, timeouts, and validation per tool call; one bad step shouldn't crash the task. **Observability:** full step-level tracing is essential to debug emergent behavior.

Step 4 — Trade-offs to Discuss

- **Multi-agent vs single agent + tools:** multi-agent helps genuinely decomposable tasks but adds latency, cost, and failure surface; often a single agent suffices.
- **LLM-driven vs deterministic orchestration:** LLM control is flexible but unpredictable; deterministic is reliable but rigid.
- **Autonomy vs guardrails:** more autonomy solves harder tasks but risks loops/cost.
- **Verifier overhead vs quality:** critics improve reliability at extra cost.

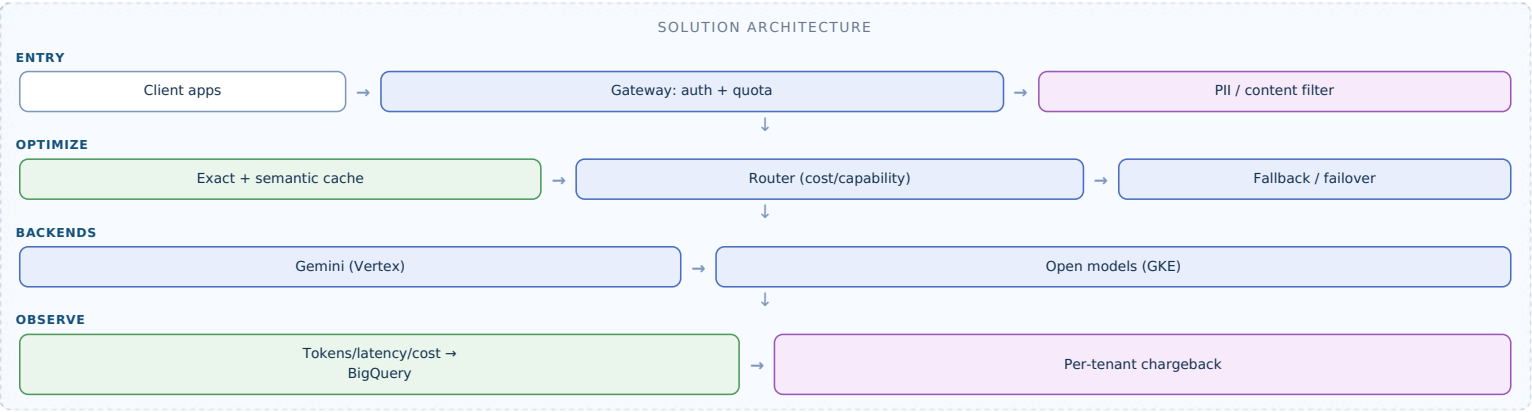
Step 5 — Common Mistake

Reaching for multi-agent because it's fashionable when a single tool-using agent would be simpler, cheaper, and more reliable — multi-agent multiplies latency, cost, and ways to fail. And when you do use it, forgetting loop/cost caps and verification, so agents chatter endlessly or compound each other's errors. Add hard limits, a verifier, and step-level tracing.

AGENT QUESTION 4

Design an LLM gateway / proxy (routing, caching, rate limiting, observability)

Focus: LLM infrastructure, model routing, cost control, reliability, governance



Step 1 — Clarify Requirements

Purpose: a single entry point for all LLM traffic across teams to control cost, enforce policy, and add reliability. **Models:** multiple (Gemini variants, open models) — do we route by cost/ capability? **Needs:** caching, rate limiting/quotas, fallbacks, prompt/PII filtering, logging, and cost attribution. **Scale:** QPS. Confirm whether it must support streaming, multi-tenant quotas, and provider failover.

Step 2 — High-Level Design

A proxy service in front of all model providers. It handles auth, per-tenant **rate limiting/quotas**, **semantic + exact caching**, **routing** (pick model by cost/capability or A/B), **fallbacks** (retry/ failover to another model on error), and **guardrails** (PII redaction, content filters). Requests/responses, tokens, latency, and cost log to BigQuery for observability and chargeback. Streaming is proxied through. Runs on **Cloud Run/GKE**; integrates Vertex (Gemini) and other backends.

Step 3 — Deep Dive

Caching: exact-match cache for identical prompts; semantic cache (embedding similarity) for near-duplicates — huge cost saver but must invalidate carefully to avoid stale answers. **Routing:** send simple queries to cheaper/smaller models, hard ones to larger; or route by tenant policy. **Reliability:** timeouts, retries with backoff, and provider failover keep the dependency from being a single point of failure. **Governance:** centralized PII filtering, content policy, prompt logging, and cost attribution per team. **Streaming:** proxy token streams without buffering the whole response.

Step 4 — Trade-offs to Discuss

- **Semantic cache hit-rate vs correctness:** aggressive semantic caching saves cost but risks returning a not-quite-right cached answer.
- **Smart routing vs simplicity:** capability/cost routing saves money but adds complexity and variance.
- **Central gateway vs direct calls:** central enforces governance but is a shared dependency — must be HA.
- **Strict guardrails vs latency:** inline filtering adds ms.

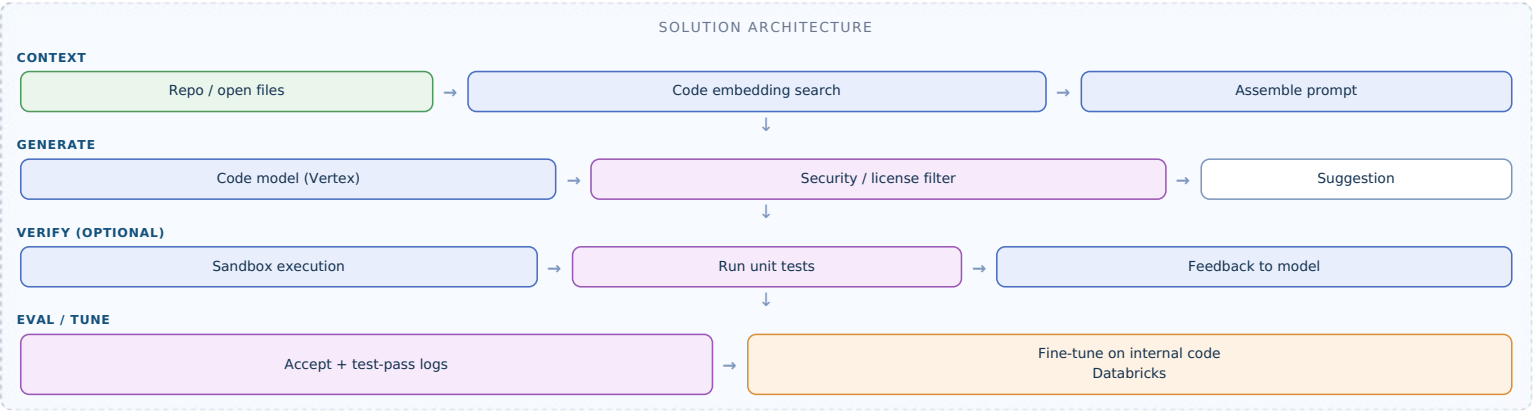
Step 5 — Common Mistake

Making the gateway a single point of failure with no fallbacks or timeouts — when one provider degrades, all LLM features go down. Build in retries, failover, and circuit breakers. Also: over-aggressive semantic caching that returns subtly wrong cached responses, and shipping with no cost attribution so teams have no incentive to use cheaper models.

AGENT QUESTION 5

Design a code-generation / coding-assistant system

Focus: Code LLM, context retrieval, execution sandbox, evaluation, safety



Step 1 — Clarify Requirements

Surface: IDE autocomplete (latency-critical, <200 ms) vs chat-based generation (seconds OK)? **Context:** how much of the repo/codebase can we retrieve for relevance? **Safety:** running generated code requires sandboxing; license/security scanning of suggestions. **Eval:** functional correctness (tests pass), not just plausibility. Confirm privacy constraints (proprietary code must not leak) and whether we fine-tune on internal code.

Step 2 — High-Level Design

A retrieval-augmented code assistant. Relevant context (open files, repo symbols, docs) is retrieved via **embedding search** over the codebase and assembled into the prompt for a code model (Gemini / Code model on Vertex). For 'run/verify' workflows, generated code executes in a **sandbox** and results feed back. Suggestions pass security/license filters. Telemetry (accept rate, test pass rate) logs for evaluation. Internal-code fine-tuning/eval happens in **Databricks**.

Step 3 — Deep Dive

Context retrieval: the quality of retrieved repo context (relevant functions, types, conventions) matters more than raw model power for codebase-specific tasks. **Latency tiers:** inline completion needs tiny latency (smaller/distilled model, short context); chat can use the big model. **Execution sandbox:** isolated, resource-capped environment to run/verify code safely — never execute untrusted output on shared infra. **Evaluation:** measure functional correctness via unit tests and real acceptance rate, not BLEU. **Privacy:** keep proprietary code in-tenant; avoid leaking to external providers.

Step 4 — Trade-offs to Discuss

- **Inline latency vs capability:** autocomplete needs speed (smaller model); chat can afford the big one.
- **More context vs cost/latency:** retrieving more repo context helps but inflates tokens.
- **Generate-only vs execute-and-verify:** execution improves correctness but needs a secure sandbox.
- **General vs fine-tuned on internal code:** fine-tuning lifts relevance but adds maintenance and privacy care.

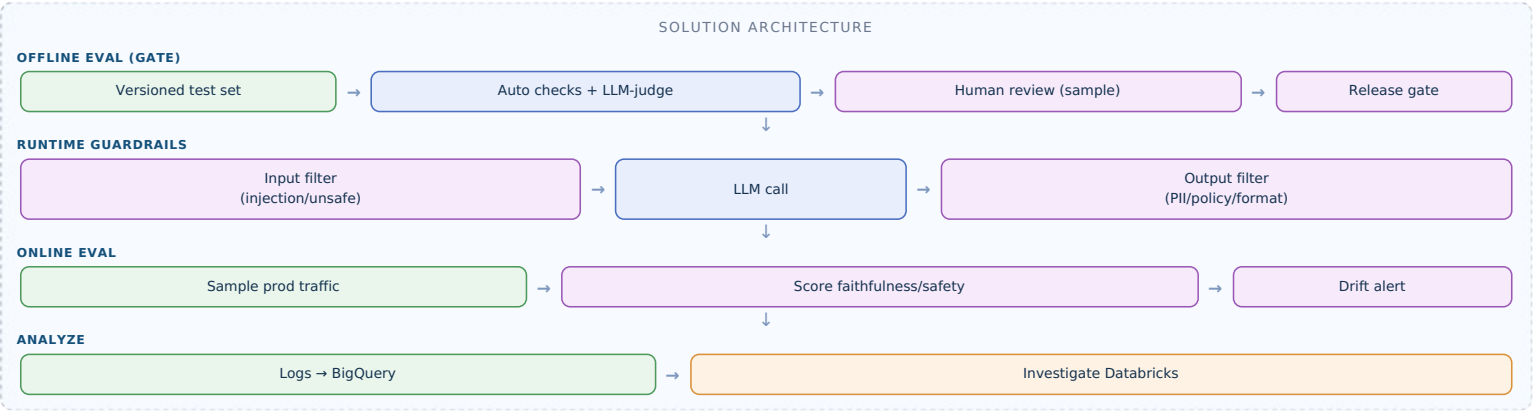
Step 5 — Common Mistake

Evaluating on surface plausibility instead of **functional correctness**, and shipping a system that suggests confident, subtly broken code. Use test-based eval and track real acceptance. The other serious mistake: executing generated code without a hardened sandbox (a security hole), or leaking proprietary code to an external model. Retrieval quality, not model size, is usually the biggest lever.

AGENT QUESTION 6

Design an LLM evaluation & guardrails framework

Focus: LLM quality assurance, automated eval, safety filters, regression testing



Step 1 — Clarify Requirements

What to evaluate: correctness, faithfulness (for RAG), safety, format, tone? **When:** offline (pre-deploy regression) and online (live monitoring)? **Labels:** do we have golden answers, or rely on LLM-as-judge / human review? **Guardrails scope:** block unsafe inputs/outputs, PII, prompt injection. Confirm we need a regression suite gating prompt/model changes, since LLM apps regress invisibly when prompts or models change.

Step 2 — High-Level Design

Two layers. **Evaluation harness:** a curated test set runs against each prompt/model version, scored by automated checks (exact/semantic match, format validators), **LLM-as-judge**, and sampled human review; results gate releases. **Runtime guardrails:** input filters (injection, unsafe content) and output filters (PII, toxicity, policy, format) wrap every call. Live traffic is sampled and scored to catch drift. Eval datasets and judge prompts are versioned. Built on Vertex + BigQuery; analysis in **Databricks**.

Step 3 — Deep Dive

Regression testing: any prompt or model change runs the eval suite; a release is blocked if quality/safety regresses — this is what stops 'we tweaked the prompt and broke 10 use cases.' **LLM-as-judge:** scalable but biased/noisy; calibrate against human labels and use it as a signal, not gospel. **Guardrails:** layered input/output filters; defend against prompt injection (especially with tools/RAG). **Online eval:** sample production traffic, score faithfulness/safety, and alert on drift. **Versioning:** treat prompts, datasets, and judges as versioned artifacts.

Step 4 — Trade-offs to Discuss

- **LLM-judge vs human eval:** judge scales cheaply but is noisy; humans are accurate but slow — blend.
- **Strict guardrails vs latency/UX:** heavy filtering adds latency and can over-block legitimate content.
- **Broad eval coverage vs maintenance:** large suites catch more regressions but cost upkeep.
- **Offline vs online eval:** offline gates releases; online catches real-world drift — need both.

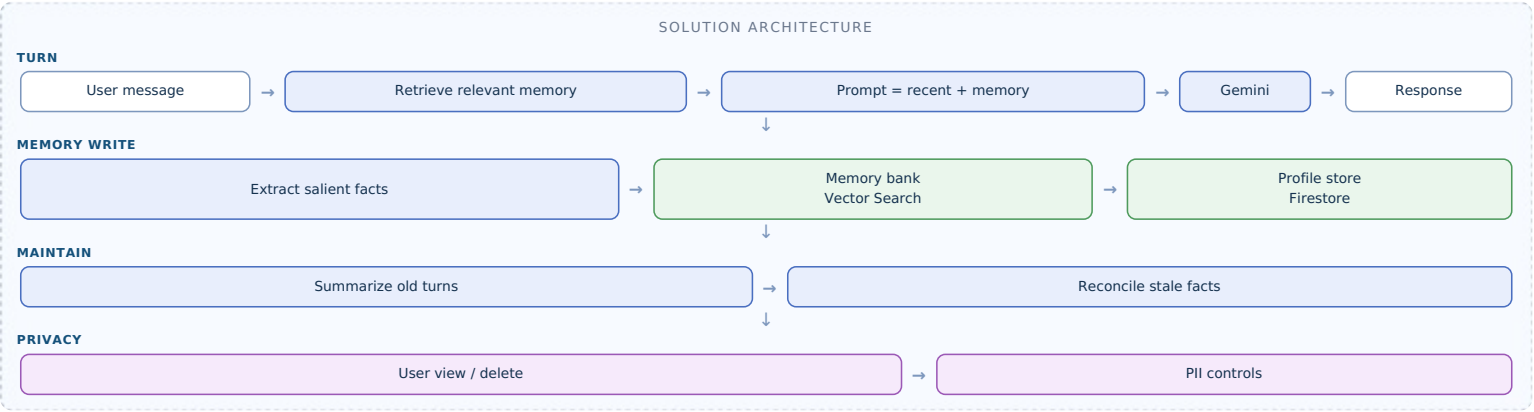
Step 5 — Common Mistake

Shipping prompt/model changes with only manual 'looks good' spot-checks and no regression suite — LLM systems regress silently, and a 'small' prompt edit breaks cases you didn't test. Build a versioned eval suite that gates releases, calibrate LLM-as-judge against humans, and wrap runtime guardrails (especially prompt-injection defenses when tools/RAG are involved).

AGENT QUESTION 7

Design a conversational agent with long-term memory

Focus: Memory architecture, summarization, retrieval, personalization, privacy



Step 1 — Clarify Requirements

Memory scope: within a session, across sessions, across users? **What to remember:** facts, preferences, past interactions? **Retrieval:** how is relevant memory recalled into context given a limited window? **Privacy:** users must be able to view/delete their memory; PII handling. Confirm we need to manage context-window limits (can't stuff all history) and avoid the agent acting on stale or wrong 'memories.'

Step 2 — High-Level Design

A memory layer separate from the LLM. **Short-term:** recent turns in the prompt. **Long-term:** salient facts/preferences extracted and stored in a **vector store (memory bank)** plus a structured profile store; at each turn, retrieve relevant memories and inject them. A summarization process compresses old conversation into durable memory. Built on Vertex (Gemini) + Vector Search + Firestore. Users can inspect and delete memories. Conflicting/old memories are reconciled.

Step 3 — Deep Dive

Context-window management: you can't keep all history in the prompt — summarize older turns and retrieve only relevant memories per query. **Memory extraction:** decide what's worth remembering (stable preferences, key facts) vs ephemeral chatter. **Retrieval:** embedding search over the memory bank surfaces pertinent items. **Reconciliation:** update or supersede outdated facts so the agent doesn't act on stale info. **Privacy:** user-visible, deletable memory with PII controls. **Personalization vs creepiness:** use memory helpfully without overreach.

Step 4 — Trade-offs to Discuss

- **Store-everything vs selective memory:** everything is simple but noisy and costly to retrieve; selective is cleaner but risks forgetting.
- **Summarization vs raw retention:** summaries save tokens but lose detail.
- **Vector recall vs structured profile:** vectors handle fuzzy recall; a structured profile is reliable for key facts — use both.
- **Personalization depth vs privacy:** richer memory aids UX but raises privacy obligations.

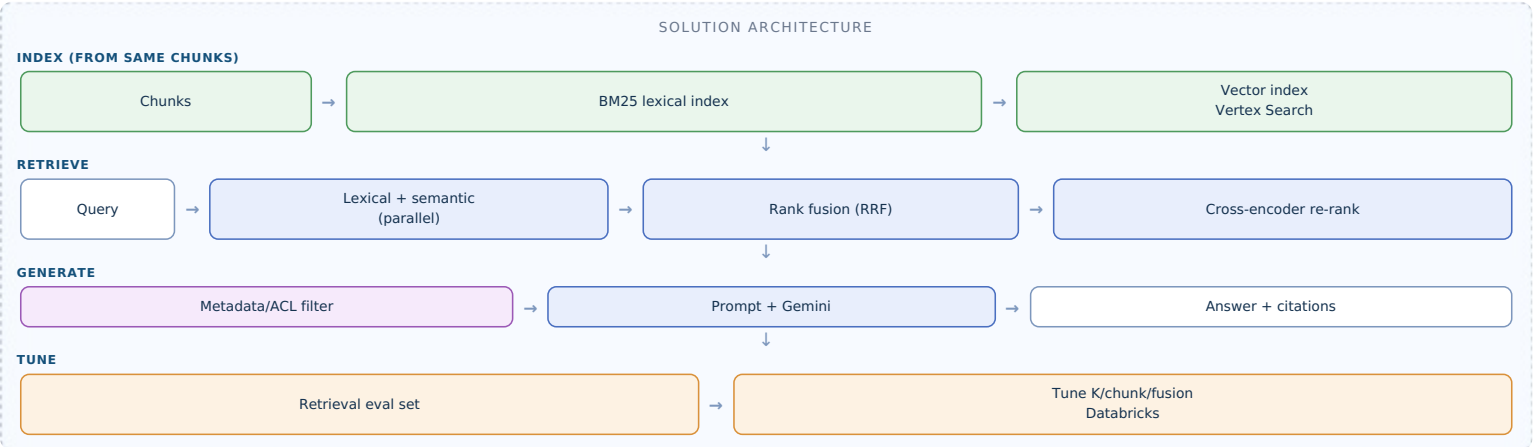
Step 5 — Common Mistake

Trying to keep entire conversation histories in the context window — it blows past limits, costs a fortune, and buries relevant info in noise. Memory must be a retrieval problem: extract salient items, store them, and recall selectively. Also: never reconciling outdated memories, so the agent confidently uses stale facts; and shipping memory with no user inspection/deletion, a privacy failure.

AGENT QUESTION 8

Design a RAG system with hybrid search (lexical + semantic + re-ranking)

Focus: Advanced retrieval, hybrid fusion, re-ranking, retrieval quality



Step 1 — Clarify Requirements

Why hybrid — pure vector search misses exact terms (codes, names, acronyms) while pure lexical misses paraphrase/intent. **Corpus traits:** lots of exact identifiers? Then lexical matters more. **Latency budget** for retrieval (<150 ms). **Quality target:** retrieval recall@K and answer faithfulness. Confirm we need metadata filtering (date, source, ACL) and that re-ranking is acceptable within the latency budget.

Step 2 — High-Level Design

A fusion retrieval pipeline. The query runs against both a **lexical index (BM25)** and a **semantic index (Vertex Vector Search)**; results are merged (e.g. reciprocal rank fusion), then a **cross-encoder re-ranker** sorts the merged set for precision. The top chunks (with metadata filtering applied) feed the LLM prompt. Ingestion builds both indexes from the same chunks. Retrieval configs are tuned offline in **Databricks** against a labeled retrieval eval set.

Step 3 — Deep Dive

Why both: lexical guarantees exact-term recall (part numbers, names); semantic captures paraphrase and intent. Fusion (RRF) combines their rankings without delicate score normalization. **Re-ranking:** a cross-encoder scores query-chunk pairs jointly for much higher precision than bi-encoder retrieval alone — the biggest quality lever. **Filtering:** apply metadata/ACL filters before or during retrieval. **Tuning:** chunk size, K per retriever, and fusion weights are tuned on a retrieval eval set — treat retrieval quality as a measurable, optimizable target, not a guess.

Step 4 — Trade-offs to Discuss

- **Hybrid vs single retriever:** hybrid improves recall across query types but doubles index cost and complexity.
- **Re-ranker precision vs latency:** cross-encoders are slow per pair; re-rank only the top candidates.
- **Fusion method:** RRF is robust and tuning-free; weighted fusion can do better with effort.
- **Index freshness vs cost:** maintaining two indexes increases update cost.

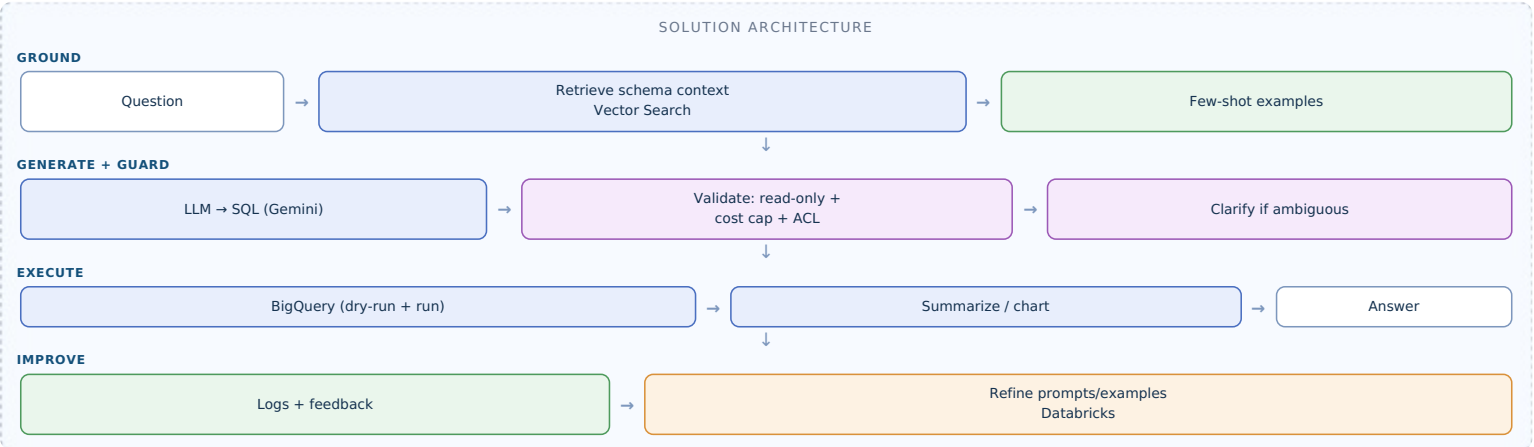
Step 5 — Common Mistake

Assuming a single vector index is always best and being surprised when it fails on exact-term queries (IDs, names, acronyms) that lexical search would nail — and skipping re-ranking, which is usually the single biggest retrieval-quality improvement available. Measure retrieval recall on a labeled set and add hybrid + re-rank where it moves the metric, rather than guessing.

AGENT QUESTION 9

Design an agentic data-analysis assistant (text-to-SQL / analytics)

Focus: Text-to-SQL, schema grounding, safe execution, verification



Step 1 — Clarify Requirements

Scope: natural-language questions over a data warehouse returning answers/charts. **Data:** how many tables, how complex the schema? **Safety:** queries must be read-only and cost-bounded (no full-table scans on petabytes). **Accuracy:** wrong SQL gives confidently wrong numbers — verification matters. Confirm we have schema metadata, that ambiguous questions can be clarified, and access controls per user/table apply.

Step 2 — High-Level Design

A grounded text-to-SQL agent. The question + retrieved **schema context** (relevant tables, columns, descriptions, sample values) is sent to an LLM (Gemini) that generates SQL; the SQL is validated (read-only, cost-limited), executed on **BigQuery**, and results are summarized/visualized. A verification step checks the query against the question; ambiguity triggers a clarifying question. Schema embeddings live in Vector Search. Logs and feedback improve prompts/examples (tuned in **Databricks**).

Step 3 — Deep Dive

Schema grounding: the model can't generate correct SQL without the right tables/columns — retrieve relevant schema (semantic search over table/column descriptions) rather than dumping the whole catalog. **Safe execution:** enforce read-only, row/byte limits, and per-user ACLs; a malformed query must not scan the warehouse or expose restricted data. **Verification:** validate syntax, dry-run for cost, and optionally have the model self-check that the SQL answers the question. **Ambiguity:** ask before guessing. **Few-shot:** curated example queries dramatically improve accuracy.

Step 4 — Trade-offs to Discuss

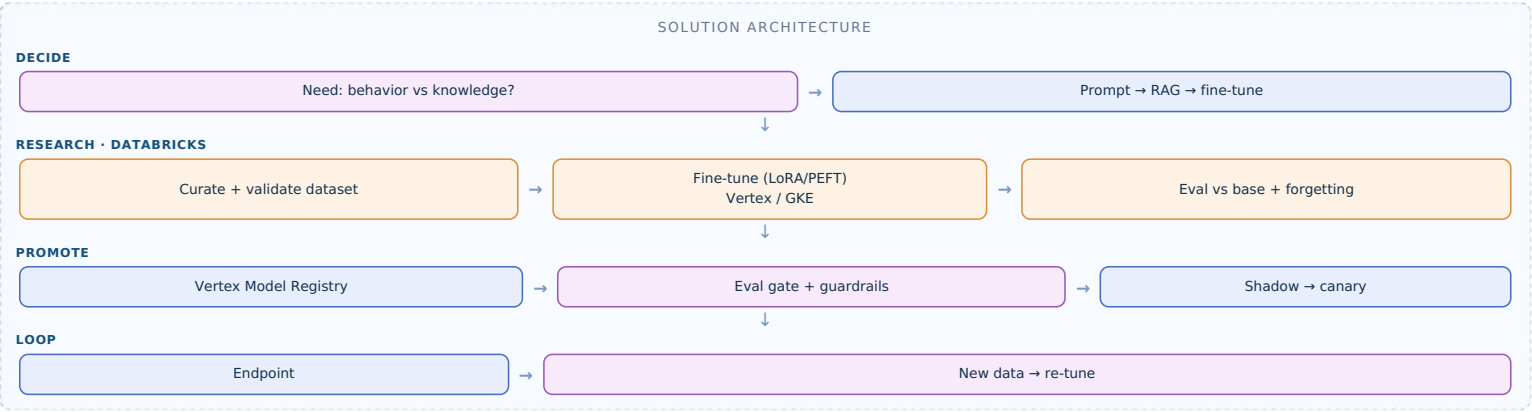
- **Full-schema vs retrieved-schema context:** retrieval scales to large schemas and improves accuracy vs dumping everything.
- **Auto-execute vs human confirm:** auto is smooth but risky; confirm for expensive/ambiguous queries.
- **Accuracy vs coverage:** restricting to known patterns is safer but less flexible.
- **Self-verification cost vs trust:** extra LLM checks cost tokens but catch errors.

Step 5 — Common Mistake

Letting the model generate and run arbitrary SQL with no schema grounding, cost limits, or verification — it confidently produces plausible-but-wrong queries (wrong joins, wrong filters) that return authoritative-looking incorrect numbers, and an unbounded query can scan a fortune of data. Ground on retrieved schema, enforce read-only + cost caps + ACLs, and verify before trusting the answer.

Design a fine-tuning / LLM customization pipeline

Focus: Fine-tuning vs RAG vs prompting, data curation, eval, deployment



Step 1 — Clarify Requirements

First question: do we even need fine-tuning, or will prompting/RAG suffice? Fine-tune for style/format/behavior or narrow tasks; use RAG for knowledge. **Data:** volume and quality of training examples; curation is the hard part. **Base model:** Gemini tuning vs an open model. **Eval:** how do we prove the tuned model is better and didn't regress/forget? Confirm hosting/cost and update cadence.

Step 2 — High-Level Design

A data-centric tuning pipeline. Curate and validate a training dataset (in **Databricks/BigQuery**), run **fine-tuning** (Vertex tuning for Gemini, or PEFT/LoRA on an open model via Vertex/GKE), evaluate against a held-out set and the base model, register the winner in **Vertex Model Registry**, and deploy to an endpoint with the standard shadow/canary rollout. An eval harness and guardrails gate promotion. The loop re-runs as new data accrues.

Step 3 — Deep Dive

Decision first: fine-tuning bakes in behavior/style and can teach narrow tasks but is the wrong tool for fresh knowledge (use RAG) — choosing wrong wastes months. **Data curation:** quality and representativeness of examples dominate outcomes; deduplicate, balance, and validate. **PEFT/LoRA:** cheaper than full fine-tuning and good enough for most adaptation. **Catastrophic forgetting:** evaluate broad capabilities, not just the target task, to ensure the model didn't lose general ability. **Promotion:** same governed research-to-prod path as any model — registry, eval gate, canary.

Step 4 — Trade-offs to Discuss

- **Fine-tune vs RAG vs prompt:** prompt first (cheap), RAG for knowledge, fine-tune for behavior/format — often combine RAG + light tuning.
- **Full fine-tune vs LoRA/PEFT:** PEFT is far cheaper with little quality loss for most cases.
- **Managed tuning vs open model:** managed is simple; open gives control, privacy, and cost options.
- **Update frequency vs stability:** frequent re-tuning tracks data but risks regressions.

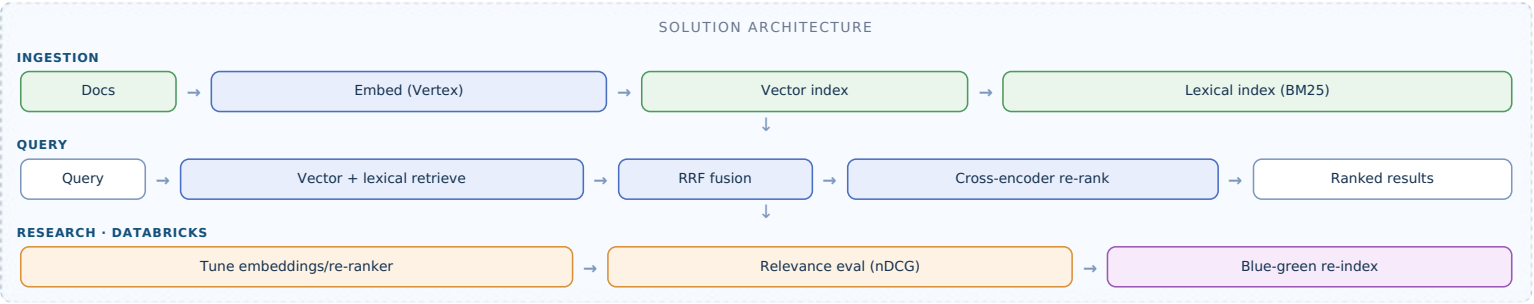
Step 5 — Common Mistake

Reaching for fine-tuning to inject knowledge that changes — you bake in facts that go stale and pay to re-tune constantly, when RAG would have solved it cleanly. Fine-tune for behavior/format, RAG for knowledge. The other classic mistake: under-investing in data curation (garbage in, garbage out) and skipping broad eval, so the tuned model quietly forgets general capabilities.

AGENT QUESTION 11

Design an LLM-powered semantic search system

Focus: Embedding search, hybrid retrieval, relevance, freshness, scale



Step 1 — Clarify Requirements

Corpus: size (millions of docs?), formats, update rate. **Query type:** short keywords, natural-language questions, or both? **Latency:** typically <200 ms p95 for search. **Relevance bar:** what does 'good' mean — click-through, downstream task success? **Scale:** QPS and concurrency. Confirm whether we need **hybrid** (lexical + semantic) rather than pure vector, and whether results must respect permissions and freshness (newly added docs searchable within seconds/minutes).

Step 2 — High-Level Design

An ingestion path embeds documents (**Vertex embeddings**) and indexes them in **Vertex Vector Search** alongside a lexical index (e.g. BM25 via an OpenSearch/Elastic-style store). At query time we run both retrievers, fuse scores (reciprocal-rank fusion), optionally re-rank the top candidates with a cross-encoder, and return ranked results with metadata. Embeddings and re-ranker are evaluated and tuned in **Databricks**, then the chosen embedding model is published to a Vertex endpoint for production encoding.

Step 3 — Deep Dive

Hybrid retrieval: pure semantic misses exact-match terms (codes, names); lexical misses paraphrase — fusing both beats either alone. **Re-ranking:** a cross-encoder over top-50 sharply improves precision at the cost of latency, so cap K. **Embedding versioning:** changing the embedding model means re-indexing the whole corpus — plan dual-index/blue-green cutover so search never breaks. **Freshness:** incremental upserts on document change. **Eval:** build a labelled relevance set and track nDCG/recall offline before shipping a new encoder; never swap embeddings blind.

Step 4 — Trade-offs to Discuss

- **Pure vector vs hybrid:** hybrid is more robust but adds a second index to operate.
- **Re-ranker on/off:** precision vs latency/cost; gate it behind a candidate cap.
- **Embedding dimension/model size:** bigger embeddings can improve quality but cost storage, RAM, and query time.
- **ANN recall vs speed:** tuning the index (ef/search params) trades recall against latency.

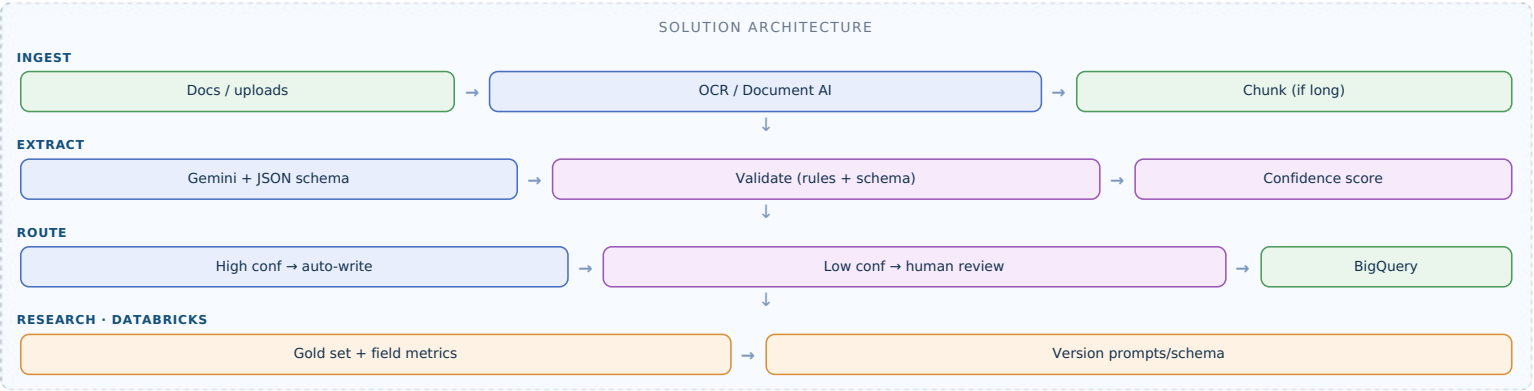
Step 5 — Common Mistake

Shipping pure semantic search and being shocked when exact-match queries (SKUs, error codes, proper nouns) return fuzzy nonsense — embeddings alone don't do precise lexical match. Add a lexical retriever and fuse. The second classic mistake: silently swapping the embedding model without re-indexing or offline eval, so half the corpus is encoded with the old model and relevance quietly collapses.

AGENT QUESTION 12

Design a document processing pipeline using LLMs (extraction + summarization)

Focus: IDP, structured extraction, summarization, validation, batch + streaming



Step 1 — Clarify Requirements

Inputs: document types (invoices, contracts, forms), volume (per day), arrival pattern (batch overnight vs real-time upload). **Outputs:** structured fields (JSON schema) and/or summaries. **Accuracy bar:** field-level precision/recall; which fields are high-stakes (money, dates) and need validation or human review? **Latency:** batch SLA vs interactive. Confirm we need schema-constrained output, confidence scoring, and a human-in-the-loop path for low-confidence extractions.

Step 2 — High-Level Design

A pipeline: ingest → OCR/parse (Document AI for scans) → chunk if long → LLM (**Gemini**) with a strict output schema (function-calling / JSON mode) for extraction or summarization → **validate** against rules and schema → route low-confidence items to human review → write structured results to a warehouse (BigQuery). Batch runs on a scheduler/Dataflow; interactive uploads hit an endpoint. Prompts and schemas are versioned; extraction quality is evaluated in **Databricks** on a labelled set.

Step 3 — Deep Dive

Schema-constrained output: force JSON / function-calling so downstream systems get parseable data, not prose; reject and retry on schema violation. **Validation:** business rules (totals add up, dates valid) catch LLM errors before they hit the database. **Confidence & HITL:** use model/uncertainty signals to auto-accept high-confidence fields and queue the rest for review — full automation on high-stakes fields is dangerous. **Long docs:** chunk + map-reduce or use long-context carefully. **Eval:** field-level metrics on a gold set; track regressions when prompts or models change.

Step 4 — Trade-offs to Discuss

- **Full automation vs HITL:** review on high-stakes fields trades throughput for safety — usually worth it.
- **Long-context vs chunk + map-reduce:** long-context is simpler but costlier and can lose mid-document detail.
- **Document AI + LLM vs LLM-only:** dedicated OCR/parse is more reliable on scans; LLM-only is simpler for clean digital docs.
- **Batch vs streaming:** batch is cheaper; streaming serves interactive needs.

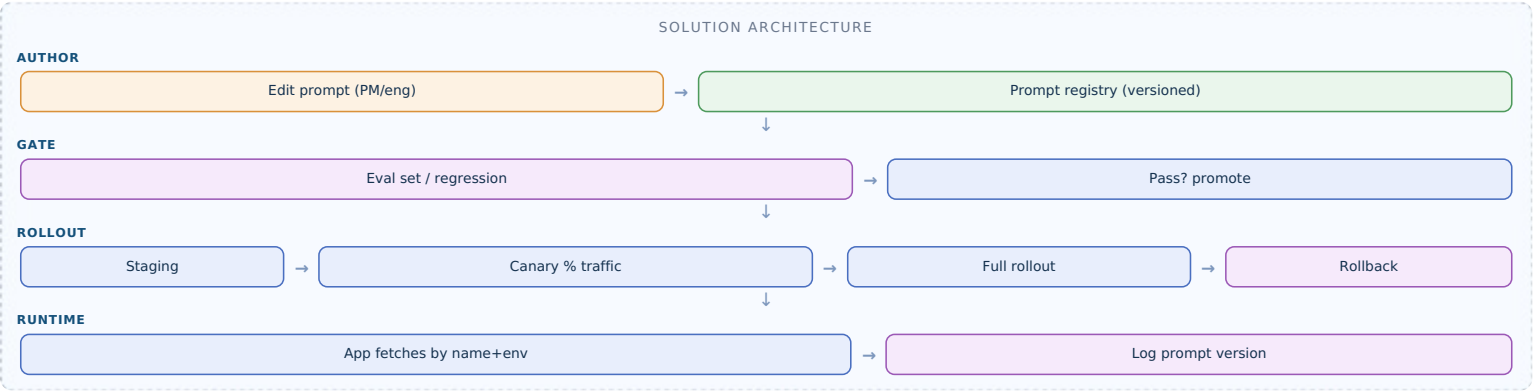
Step 5 — Common Mistake

Trusting raw LLM output as structured data with no schema enforcement or validation — a single malformed field or hallucinated total flows straight into a financial system. Always constrain output to a schema, validate with business rules, attach confidence, and keep humans on the high-stakes fields. The other mistake: evaluating on 'looks right' instead of field-level precision/recall on a labelled gold set.

AGENT QUESTION 13

Design a prompt management and versioning system

Focus: Prompt registry, versioning, A/B testing, rollout, eval gating



Step 1 — Clarify Requirements

Who edits prompts: engineers, PMs, prompt specialists? **Scale:** how many prompts, across how many features/models? **Change cadence:** how often do prompts change, and how risky is a bad change? **Requirements:** version history, rollback, environment separation (dev/staging/prod), and the ability to A/B a new prompt before full rollout. Confirm we want prompts decoupled from code deploys (edit without shipping) but still governed by evaluation, not edited live in production unchecked.

Step 2 — High-Level Design

A **prompt registry** stores prompts as versioned, named artifacts with metadata (model, params, owner, eval scores). Applications fetch a prompt by name + environment label at runtime (with caching), rather than hard-coding strings. A change goes: edit → run against an **eval set** (in Databricks / an eval harness) → if it passes, promote to staging, then canary a percentage of prod traffic, then full rollout — with instant rollback to the previous version. All inference logs the prompt version used.

Step 3 — Deep Dive

Prompts are code: they change model behavior as much as a code change, so they need versioning, review, eval, and staged rollout — not live edits. **Eval gating:** every candidate prompt runs against a regression set; block promotion on score drop. **A/B + canary:** route a slice of traffic to the new prompt, compare quality/cost/latency, then ramp. **Version pinning & logging:** log which prompt version produced each output so you can debug and attribute regressions. **Rollback:** one click back to a known-good version. **Decoupling:** editing a prompt shouldn't require a full app redeploy.

Step 4 — Trade-offs to Discuss

- **Decouple prompts from code vs simplicity:** a registry adds infrastructure but enables safe, fast iteration.
- **Self-serve editing vs governance:** letting PMs edit speeds iteration but needs eval gates to stay safe.
- **Aggressive vs conservative rollout:** fast ramp iterates quicker; canary protects quality.
- **Centralized registry vs per-service prompts:** central gives consistency/observability; per-service gives autonomy.

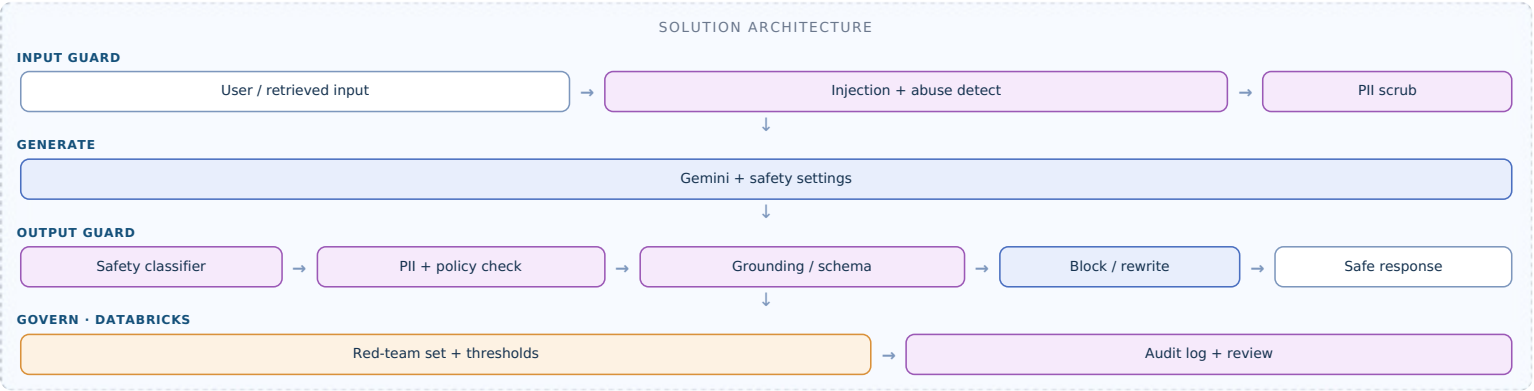
Step 5 — Common Mistake

Treating prompts as throwaway strings edited live in production with no versioning, eval, or rollback — someone tweaks a prompt, quality silently degrades, and nobody can tell what changed or revert it. Prompts are behavior-defining artifacts: version them, gate changes on evaluation, roll out via canary, log the version on every call, and keep one-click rollback.

AGENT QUESTION 14

Design a safety and content-moderation layer for LLM outputs

Focus: Guardrails, input/output filtering, jailbreak defense, policy, audit



Step 1 — Clarify Requirements

Surface: consumer chatbot, internal tool, or API for third parties? Risk profile differs. **Policies:** what's disallowed (PII leakage, harmful content, off-topic, competitor mentions)? **Latency budget:** moderation adds overhead — how much is acceptable? **Failure mode preference:** fail-closed (block on uncertainty) vs fail-open? Confirm we need both **input** (prompt-injection/jailbreak, abusive input) and **output** (toxic, unsafe, PII, off-policy) checks, plus an audit trail for every block.

Step 2 — High-Level Design

A guardrail layer wraps the model: **input filters** (moderation classifier, prompt-injection detection, PII scrubbing) run before inference; the LLM call happens; then **output filters** (safety classifier, PII detection, policy/grounding checks, schema validation) run before the response reaches the user. Violations are blocked or rewritten and logged with reason. Use managed safety filters (Vertex safety settings) plus custom classifiers and rules. Policies and thresholds are tuned in **Databricks** against a labelled red-team set.

Step 3 — Deep Dive

Defense in depth: no single filter is perfect, so layer model-native safety settings, a moderation classifier, regex/PII rules, and grounding checks. **Prompt injection:** treat retrieved/user content as untrusted; separate instructions from data and detect override attempts — the top agent-era risk. **Fail-closed:** on detector uncertainty for high-risk surfaces, block rather than pass. **Latency:** run cheap filters first, escalate to expensive ones only as needed; some checks can run in parallel with generation. **Audit:** log every decision with the triggering policy so you can review, tune thresholds, and prove compliance.

Step 4 — Trade-offs to Discuss

- **Strict vs permissive:** tighter filters reduce harmful output but increase false positives/over-refusal and hurt UX.
- **Latency vs coverage:** more checks are safer but slower; order and parallelize.
- **Managed safety vs custom classifiers:** managed is fast; custom catches domain-specific policy.
- **Block vs rewrite:** blocking is safe but blunt; rewriting preserves UX at added complexity.

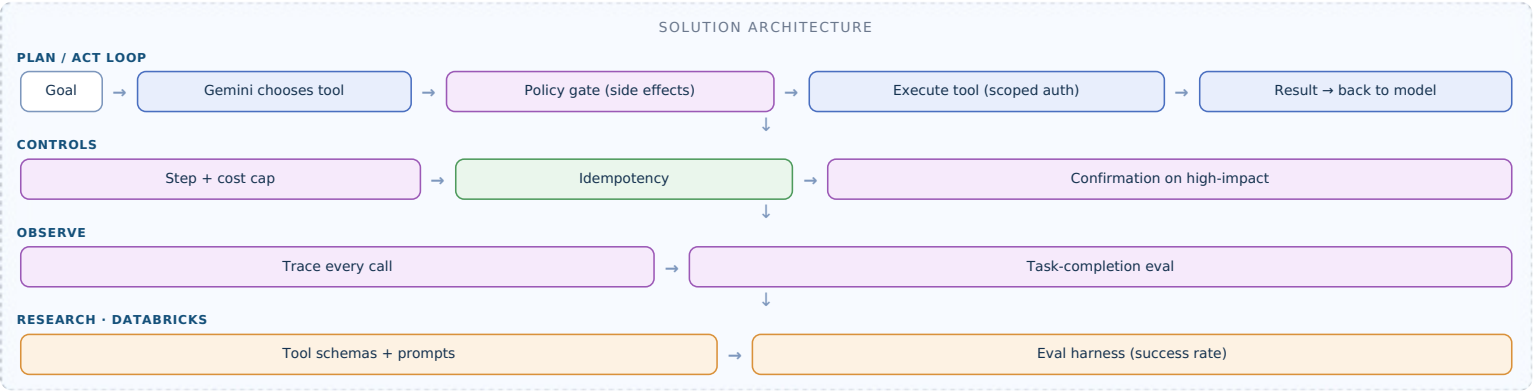
Step 5 — Common Mistake

Only filtering output and ignoring input — or trusting the model's built-in safety alone. Without input-side prompt-injection defense, retrieved or user content hijacks the agent; without layered output checks, one classifier miss ships harmful or PII-leaking content. Guardrails must be defense-in-depth on both sides, fail-closed on high-risk surfaces, and fully audited — not a single bolt-on filter.

AGENT QUESTION 15

Design a tool-using agent for workflow automation (function calling)

Focus: Function calling, planning, tool routing, reliability, authorization



Step 1 — Clarify Requirements

Tasks: what workflows (e.g. create tickets, query systems, send emails)? **Tools:** which APIs, and which are read-only vs state-changing? **Autonomy:** fully autonomous or human-approval on side-effects? **Reliability bar:** what's the cost of a wrong action? **Auth:** the agent must act with the right, scoped permissions per user. Confirm we need bounded tool access, confirmation on high-impact actions, idempotency, and full traceability of every tool call the agent makes.

Step 2 — High-Level Design

An orchestration loop (Vertex Agent Builder or custom on **Gemini** with function calling): the model receives the goal plus a registry of tool schemas, decides which tool to call with which arguments, the runtime executes the tool (with auth + validation), feeds the result back, and the loop continues until the task is done or it asks for help. A **policy layer** gates side-effecting tools (confirmation, limits). Every step is logged. Tool definitions, prompts, and the eval harness are developed in **Databricks**.

Step 3 — Deep Dive

Tool design: clear schemas and descriptions drive correct selection — ambiguous tools cause wrong calls. **Authorization:** execute tools with the user's scoped credentials, never broad service-account power; validate arguments server-side. **Side-effect safety:** require confirmation or dry-run for destructive actions; make operations idempotent so a retry doesn't double-charge. **Loop control:** cap steps and cost to prevent runaway loops; detect when the agent is stuck. **Error handling:** feed tool errors back so the agent can recover. **Eval:** test on task-completion success, not just plausible-looking plans.

Step 4 — Trade-offs to Discuss

- **Autonomy vs control:** more autonomous agents do more but risk costly wrong actions; gate side effects.
- **Many fine-grained tools vs few broad ones:** fine-grained is safer/clearer but more to maintain.
- **Planner-executor vs single loop:** explicit planning helps complex tasks at added complexity.
- **Model-driven vs deterministic orchestration:** deterministic flows are reliable; model-driven is flexible but less predictable.

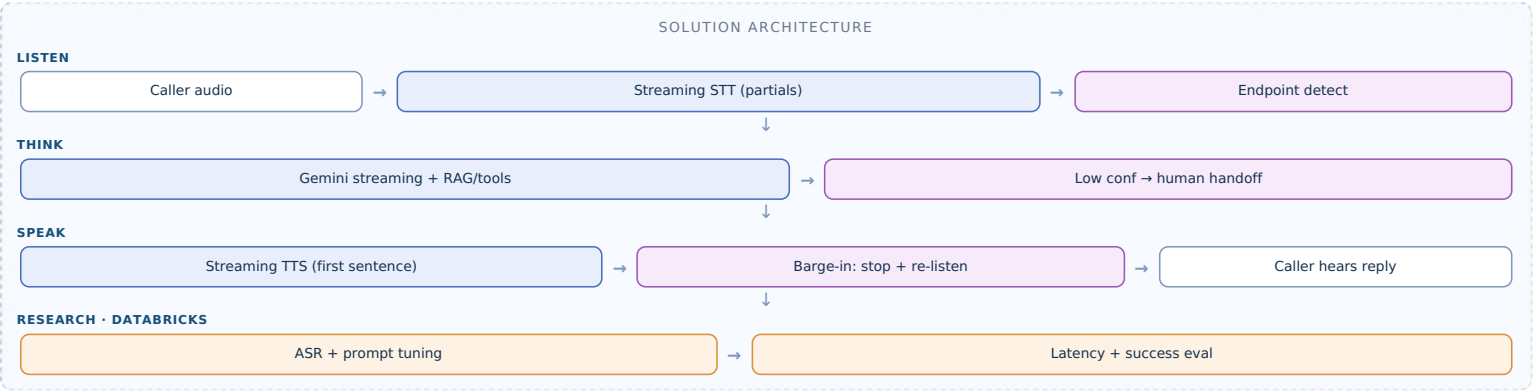
Step 5 — Common Mistake

Giving the agent broad, unscoped credentials and letting it execute state-changing tools autonomously with no confirmation, idempotency, or step/cost caps — one bad plan deletes data, sends wrong emails, or loops forever burning tokens. Scope permissions per user, gate destructive actions, make tools idempotent, bound the loop, and log every call. Evaluate on real task completion, not nice-looking reasoning traces.

AGENT QUESTION 16

Design a real-time voice AI agent

Focus: Streaming STT, LLM, TTS, latency budget, barge-in, turn-taking



Step 1 — Clarify Requirements

Use case: phone support, voice assistant, drive-through? **Latency:** conversational feel needs end-to-end response <~1 s — this dominates the design. **Languages/accent**s and noise conditions. **Capabilities:** Q&A only or actions/tool calls too? **Interruptions:** must the user be able to barge in mid-response? Confirm we need streaming STT (partial transcripts), streaming LLM + TTS, and graceful turn-taking, plus escalation to a human.

Step 2 — High-Level Design

A streaming pipeline: audio in → **streaming STT** (Google Speech-to-Text) producing partial transcripts → endpoint detection → **Gemini** (streaming) with RAG/tools for the reply → **streaming TTS** back to the caller, starting playback on the first sentence. A dialog manager handles turn-taking and barge-in (stop TTS when the user speaks). Tools/RAG ground answers; low confidence triggers human handoff. Latency is engineered at every hop. Prompts, eval, and ASR/quality tuning happen in **Databricks**.

Step 3 — Deep Dive

Latency is the product: stream every stage and overlap them — begin LLM generation before the user fully stops, begin TTS on the first chunk; never wait for full transcripts/responses. **Endpointing:** detect end-of-turn well, or the agent talks over people or pauses awkwardly. **Barge-in:** immediately stop TTS and re-listen when the user interrupts. **Robust STT:** noise, accents, and domain terms hurt recognition; bias with phrase hints. **Grounding + handoff:** RAG for answers, escalate on low confidence. **Eval:** measure latency percentiles and task success, not just transcription WER.

Step 4 — Trade-offs to Discuss

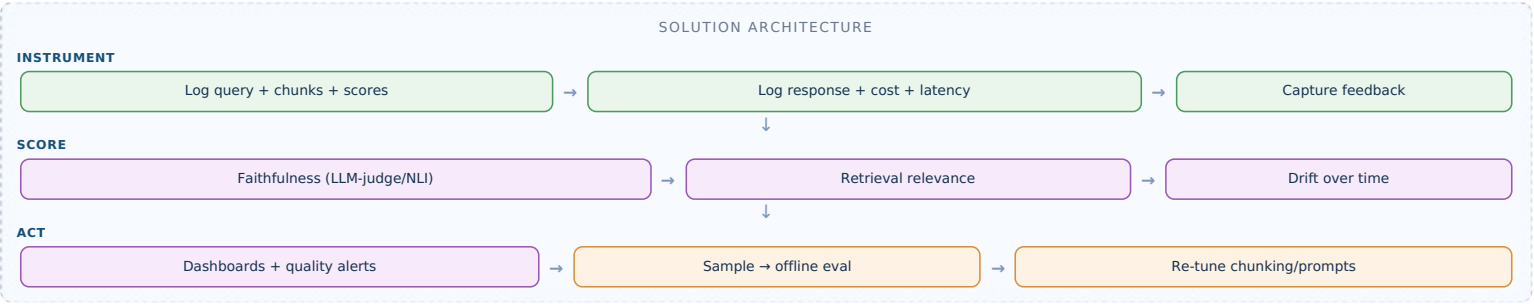
- **Latency vs quality:** a bigger model reasons better but blows the time budget; balance or use a fast model.
- **Cascaded (STT→LLM→TTS) vs speech-native:** cascaded is modular/debuggable; native can be lower latency but less controllable.
- **Eager vs careful endpointing:** eager feels snappy but risks cutting users off.
- **On-device vs cloud:** on-device cuts latency/privacy risk; cloud gives capability.

Step 5 — Common Mistake

Building it as a non-streaming request/response pipeline — wait for the full transcript, then the full LLM answer, then synthesize all the audio — producing multi-second dead air that feels broken. Voice must stream and overlap stages, handle barge-in, and nail endpointing. Optimizing transcription accuracy while ignoring end-to-end latency percentiles is the same mistake in disguise.

Design RAG observability and quality monitoring

Focus: LLM observability, faithfulness, retrieval quality, online eval, feedback



Step 1 — Clarify Requirements

What we monitor: retrieval quality, answer faithfulness (grounding), latency, cost, user satisfaction. **Signals available:** explicit feedback (thumbs), implicit (follow-ups, abandonment), and offline labels. **Volume:** traffic and how much we can afford to LLM-judge. **Alerting:** what regression should page someone? Confirm the goal is to catch silent quality decay — the model still answers fluently but starts hallucinating or retrieving poorly — which standard infra metrics never reveal.

Step 2 — High-Level Design

Instrument the RAG pipeline to log, per request: query, retrieved chunks + scores, prompt, response, latency, cost, and feedback. A monitoring layer computes quality signals: retrieval metrics (were relevant chunks found?), **faithfulness** (does the answer follow from the context, scored by an LLM-judge or NLI model), and user-feedback rates. Dashboards + alerts track these over time; sampled traffic is sent for deeper offline eval in **Databricks**, where regressions trigger investigation and re-tuning of chunking/retrieval/prompts.

Step 3 — Deep Dive

Fluent ≠ correct: LLM outputs always look plausible, so token-level infra metrics tell you nothing about quality — you must measure faithfulness and retrieval relevance explicitly. **LLM-as-judge:** score grounding/relevance on sampled traffic; calibrate the judge against human labels so you trust it. **Feedback loop:** capture thumbs and implicit signals, and feed hard cases back into the eval set. **Drift detection:** watch faithfulness, retrieval scores, and refusal/abstain rates over time. **Cost/latency:** track per-query token cost. **Trace everything:** full request traces make debugging a bad answer possible.

Step 4 — Trade-offs to Discuss

- **Sampling rate for LLM-judge:** judging everything is costly; sample, but rare failures may slip.
- **LLM-judge vs human eval:** judges scale cheaply but can be biased; humans are truth but slow.
- **Online vs offline eval:** online catches real issues fast; offline is rigorous and reproducible — do both.
- **Alert sensitivity:** tight thresholds catch decay early but page on noise.

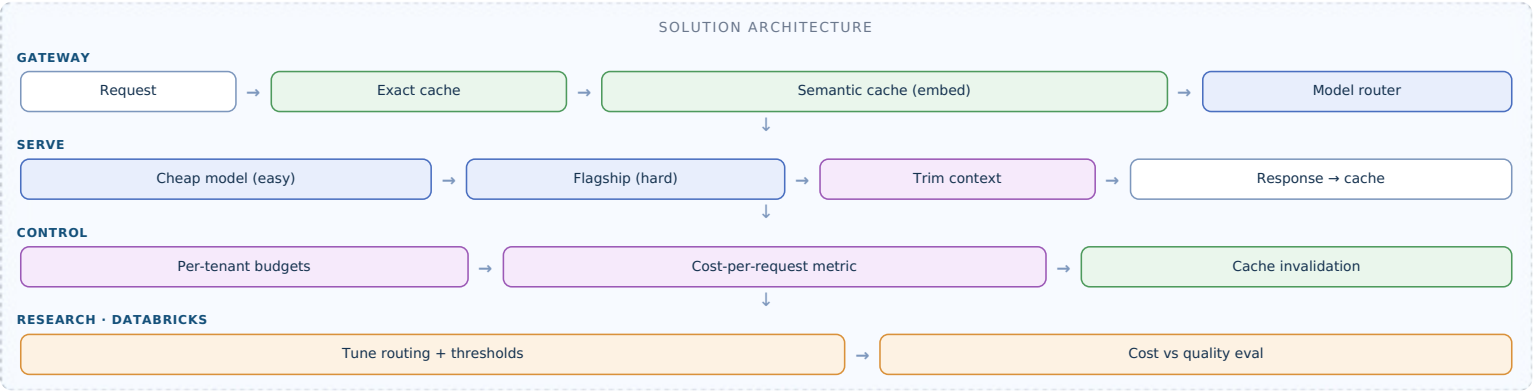
Step 5 — Common Mistake

Monitoring only latency, error rate, and throughput — the system looks perfectly healthy while answer quality silently rots because retrieval drifted or a prompt/model change hurt grounding. Fluency hides correctness. You must measure faithfulness and retrieval quality (LLM-judge calibrated to humans), capture feedback, trace requests, and alert on quality regressions, not just infra signals.

AGENT QUESTION 18

Design an LLM cost-optimization and caching layer

Focus: Caching, model routing, token economics, semantic cache, budgets



Step 1 — Clarify Requirements

Cost drivers: request volume, average tokens in/out, model mix. **Workload shape:** repeated / similar queries (cacheable) vs all-unique? **Quality floor:** how much quality can we trade for cost? **SLA:** latency limits that constrain options. Confirm whether requests repeat enough for caching to pay off, whether many requests are simple enough to route to a cheaper model, and that we need per-tenant budgets/limits to prevent runaway spend.

Step 2 — High-Level Design

A gateway sits in front of the models and applies, in order: an **exact cache** (identical prompt→cached response), a **semantic cache** (embed the query, reuse a near-duplicate's answer above a similarity threshold), **model routing** (send simple requests to a cheap/small model, hard ones to a large one), prompt-context trimming, and **budgets/rate-limits** per tenant. Misses go to the model; responses populate the caches. Routing rules and quality/cost trade-offs are evaluated in **Databricks** before rollout.

Step 3 — Deep Dive

Caching: exact caching is free wins on repeats; semantic caching extends that to paraphrases but risks returning a subtly wrong answer if the threshold is loose — tune carefully and scope per context. **Model routing:** a cheap classifier or heuristics decide difficulty; most traffic is easy and doesn't need the flagship model. **Prompt economy:** trim retrieved context and history; tokens are the bill. **Budgets:** per-tenant quotas and alerts stop one customer or a bug from causing a spend spike. **Measure:** track cost per request and ensure optimizations don't quietly degrade quality.

Step 4 — Trade-offs to Discuss

- **Semantic cache hit-rate vs correctness:** looser thresholds save more but risk wrong reuse.
- **Cheap-model routing vs quality:** routing saves cost but mis-routing a hard query hurts UX.
- **Context trimming vs answer quality:** fewer tokens is cheaper but can drop needed info.
- **Cache freshness:** caching stale answers when underlying data changes — needs invalidation.

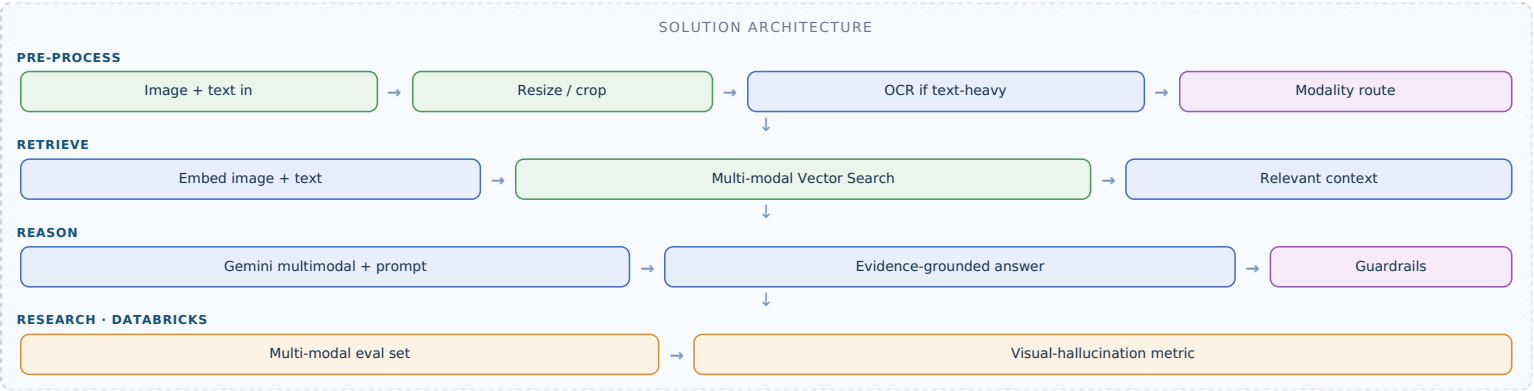
Step 5 — Common Mistake

Bolting on a semantic cache with a loose similarity threshold and no invalidation — users start getting confidently wrong, stale, or mismatched answers because a 'similar' query isn't the same query. Cost optimization must never silently degrade quality: tune cache thresholds, scope and invalidate caches, route by real difficulty, and continuously check that quality holds while cost drops.

AGENT QUESTION 19

Design a multi-modal agent (text + image understanding)

Focus: Vision-language, multi-modal RAG, grounding, OCR, modality routing



Step 1 — Clarify Requirements

Inputs: what images (photos, screenshots, charts, documents) and paired text? **Tasks:** visual Q&A, describe, extract, compare, or act on what's seen? **Latency & volume.** **Quality bar:** how costly is a misread image? Confirm we need a vision-language model (Gemini multimodal), whether we must **retrieve** over images/diagrams too (multi-modal RAG), and how to handle images with dense text (OCR) versus purely visual content — the routing matters.

Step 2 — High-Level Design

A multi-modal orchestrator on **Gemini**: inputs (image + text) are pre-processed (resize, and run OCR when the image is text-heavy), then sent to the vision-language model with the task prompt. For knowledge-grounded tasks, add **multi-modal RAG** — embed images/diagrams and text into a shared or parallel index in Vector Search and retrieve relevant context. Tool calls and guardrails apply as in any agent. Outputs are grounded and cited where possible. Embedding/prompt strategy is tuned in **Databricks**.

Step 3 — Deep Dive

Modality routing: a document photo full of text is best served by OCR + text reasoning; a chart or scene needs the vision model — routing wrong wastes the model's strength. **Grounding:** for visual Q&A, make the model reference specific image regions/evidence to reduce hallucination. **Multi-modal retrieval:** aligning image and text embeddings is hard; often retrieve text describing images plus the images themselves. **Pre-processing:** resolution and cropping materially affect accuracy. **Eval:** build a labelled multi-modal set; visual hallucination (confidently describing things not in the image) is the key failure to measure.

Step 4 — Trade-offs to Discuss

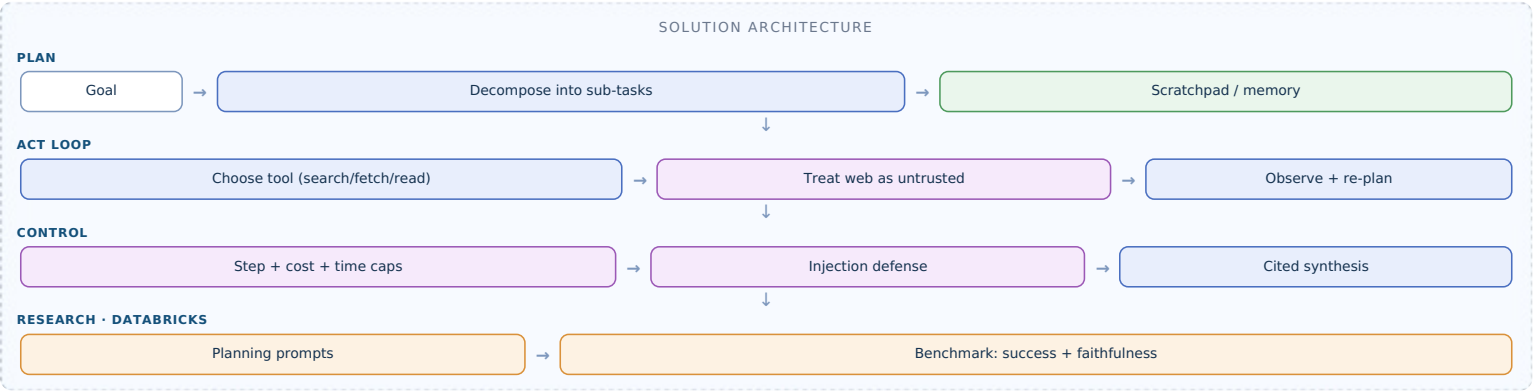
- **Single multimodal model vs pipeline (OCR/detector + LLM):** one model is simpler; a pipeline is more accurate on text-heavy or specialized images.
- **Image resolution vs cost/latency:** higher res helps accuracy but costs more tokens/time.
- **Unified vs separate embedding spaces for retrieval:** unified is elegant but harder to get right.
- **Describe-then-reason vs direct:** captioning first aids grounding but can lose detail.

Step 5 — Common Mistake

Throwing every image straight at the vision model and trusting the description — for text-dense images (forms, screenshots) it misreads numbers and details that proper OCR would nail, and it confidently hallucinates content that isn't there. Route by image type (OCR text-heavy images), demand evidence-grounded answers, pre-process resolution, and evaluate specifically for visual hallucination on a labelled set.

Design an autonomous research / web-browsing agent

Focus: Autonomous agents, planning, web tools, reliability, loop control, safety



Step 1 — Clarify Requirements

Goal type: answer research questions, gather data, or take actions on the web? **Sources:** open web, internal docs, APIs? **Autonomy & oversight:** fully autonomous or checkpoints? **Budget:** max steps/cost/time per task — unbounded agents are dangerous. **Trust:** how do we handle unreliable or adversarial web content (injection)? Confirm we need planning over multiple steps, source verification, strict loop bounds, and safety controls so the agent can't be hijacked by a malicious page or loop forever.

Step 2 — High-Level Design

A planner-executor agent on **Gemini**: decompose the goal into sub-tasks, then iterate — choose a tool (web search, fetch page, read internal doc, calculator), execute it, observe, update the plan, and synthesize a cited result. A **memory/scratchpad** holds intermediate findings. A **controller** enforces step/cost/time budgets and treats web content as untrusted (injection defense). Results carry citations to sources. Prompts, planning strategy, and an eval harness on benchmark research tasks are developed in **Databricks**.

Step 3 — Deep Dive

Planning & recovery: long-horizon tasks need decomposition and the ability to re-plan when a step fails or returns junk — brittle linear plans break. **Loop control:** hard caps on steps, cost, and time plus stuck-detection prevent runaway/expensive loops. **Untrusted content:** a fetched page may contain prompt-injection ('ignore your instructions') — isolate data from instructions and never let page text issue commands. **Source grounding:** verify across sources and cite, so the user can check claims. **Eval:** benchmark on task success and faithfulness, not on producing confident-sounding reports.

Step 4 — Trade-offs to Discuss

- **Autonomy vs reliability/cost:** more autonomous agents tackle harder tasks but fail unpredictably and burn budget; bound them.
- **Depth vs budget:** deeper exploration finds more but costs more time/tokens.
- **Open web vs curated sources:** open web is broad but noisy and adversarial; curated is safer/narrower.
- **Single agent vs planner-executor split:** explicit planning helps complex tasks at added complexity.

Step 5 — Common Mistake

Letting the agent run unbounded over the open web with no step/cost caps, no injection defense, and no source verification — it loops up huge bills, gets hijacked by a malicious page that overrides its instructions, and produces a confident report full of unverified or fabricated claims. Bound the loop, isolate untrusted content from instructions, verify and cite sources, and evaluate on real task success, not eloquence.